

— PHASE 4: DECEPTION GRID + DIGITAL TWIN + SIEM —
→ 4.1 → 4.1b [NEW v3] → 4.2 → 4.3

— PHASE 5: ZERO TRUST ARCHITECTURE [ALL NEW v3] —
→ 5.1 → 5.2 → 5.3 → 5.4 → 5.5

— PHASE 6: PROACTIVE INTELLIGENCE + WEB CRAWLING [ALL NEW v3] —
→ 6.1 → 6.2 → 6.3 → 6.4

— PHASE 7: NETWORK TRAFFIC ANALYSIS + ENDPOINT [ALL NEW v3] —
→ 7.1 → 7.2 → 7.3

— PHASE 8: THREAT INTELLIGENCE PLATFORM [ALL NEW v3] —
→ 8.1 → 8.2 → 8.3

— PHASE 9: UEBA + BEHAVIOURAL ANALYTICS [ALL NEW v3] —
→ 9.1 → 9.2

— PHASE 10: CASE MANAGEMENT + PLAYBOOKS [ALL NEW v3] —
→ 10.1 → 10.2 → 10.3

— PHASE 11: MULTI-TENANCY + FORENSICS [ALL NEW v3] —
→ 11.1 → 11.2

— PHASE 12: SUPPLIER TRUST + FEED INTEGRITY [ALL NEW v3] —
→ 12.1 → 12.2

— VALIDATION —
→ VALIDATION (expanded from v2.0)

CONTEXT INJECTION (paste this at the start of EVERY session)

You are working on Kebos AI – a full-stack post-quantum-native Cyber Threat Platform
purpose-built for the Indian BFSI, Government SOC, and Defence Contractor market.

CODEBASE STRUCTURE:

- kebos-backend/: FastAPI 0.104.1 / Python 3.11, PostgreSQL + TimescaleDB + InfluxDB,
Kafka + Redis + Celery, Alembic migrations

Modules: auth/, audit_logger/, job_manager/, threat_detection/,
siem_integration/,
network_analytics/, messaging/, genai_assistant/, dashboard/, security/,
deception/, simulation/, reporting/, integrations/, crawlers/, nta/, tip/,
ueba/, cases/, playbooks/
Auth: JWT RS256 + bcrypt + RBAC (ADMIN, AUDITOR, ANALYST, OPERATOR, DEVELOPER)
Real-time: WebSocket + Socket.io
AI: CatBoost (primary threat detector) + Gemma LLM (tiered routing)

- kebos-frontend/: React 18 + TypeScript + Vite + Tailwind + Zustand + React Query
+ Socket.io

Auth state via HttpOnly cookies (NOT localStorage)

- qmind_enterprise/: Python 3.12 + FastAPI, real PQC architecture:
Kyber-768 (FIPS 203) + Dilithium-3 (FIPS 204) + AES-256-GCM
USE_REAL_PQC=true in production (verify at startup, never claim PQC if false)
10-category probabilistic engine + signal decay + adversarial stability
8 live external feeds + supplier trust scoring engine
Kafka topics: threat.indicators, qmind.results, honeypot.interactions,
audit.signed, crawler.discoveries, analyst.feedback

DATA CLASSIFICATION (governs every LLM call):

PUBLIC – IOC categories, feed names → Groq/Gemini API allowed
INTERNAL – QMind scores, advisories → External LLM with sanitisation
CONFIDENTIAL – Customer IPs, hostnames, users → Local Gemma only
RESTRICTED – Govt SOC data, classified IOCs → Air-gapped Gemma only

COMPLIANCE:

CERT-In 6-hour mandatory reporting window (hard legal deadline)
DPDPA: all customer data in ap-south-1 Mumbai, never leaves India
RBI Cybersecurity Framework: network-level monitoring required for BFSI

PHILOSOPHY:

Advisory-first. Never auto-block. REVERSIBLE actions auto-execute + notify.
IRREVERSIBLE actions: Digital Twin dry-run → analyst approval → execute.
Proactive detection badge required on all crawler/CT-log discovered threats.
Supplier trust scoring applied to all 8 external feeds at all times.

CRITICAL BUGS (fixed in v2.0, must remain fixed):

- JWT HttpOnly cookie, RS256, 15-min expiry, Redis JTI blacklist
- aiokafka throughout (never kafka-python)
- qmind.results consumer running in Kebos backend
- /signals/inject endpoint live
- SOCReportGenerator LLM client wired at startup

- `_parse_report_sections` uses JSON-mode (not line-prefix parsing)
- `slowapi` with Redis backend (not in-memory)
- `asyncio.create_task()` has `done_callback` everywhere
- Docker socket replaced by Tecnativa proxy in HoneyGrid
- syslog over TCP+TLS (never UDP)
- TOTP secrets encrypted via Vault transit (never plaintext)

PHASE 1 — AUTH HARDENING

Prompt 1.1 — JWT Migration to HttpOnly Cookie

Read the full contents of:

- `kebos-frontend/src/api/apiClient.ts`
- `kebos-frontend/src/store/` (all files)
- `kebos-backend/app/auth/router.py`
- `kebos-backend/app/auth/services.py`
- `kebos-backend/app/main.py`

Then make ALL of the following changes:

1. In `apiClient.ts`:

- Remove every instance of `localStorage.setItem('auth_token', ...)`
- Remove every instance of `localStorage.getItem('auth_token')`
- Remove every instance of `localStorage.removeItem('auth_token')`
- Replace token injection in request headers with: `credentials: 'include'` on every `fetch/axios` call so the browser sends the HttpOnly cookie automatically

- Do NOT manually attach `Authorization` headers – the cookie carries the session

2. In `kebos-backend/app/auth/router.py`, find the login endpoint:

- After successful authentication, instead of returning token in JSON body, set a cookie: `response.set_cookie(`
 `key="access_token",`
 `value=f"Bearer {token}",`
 `httponly=True,`
 `samesite="strict",`
 `secure=True,`
 `max_age=900` # 15 minutes – enforced globally
 `)`
- Add a `GET /auth/me` endpoint that reads `access_token` from `request.cookies`, validates the JWT, and returns the current user object


```
3. In kebos-backend/app/main.py:
    - Import and call validate_environment() from security.py at application
      startup,
      BEFORE the app starts accepting requests
    - If validate_environment() returns any errors, log them and raise
      SystemExit(1)
    - Register get_security_headers() as middleware

4. In kebos-frontend/src/store/:
    - Remove any state that stores the raw JWT string
    - Replace with a boolean isAuthenticated and a user profile object
    - On app load, call GET /auth/me to rehydrate auth state from the cookie

Do not change any other files. Show me the complete diff for review.
```

Prompt 1.2 — RS256 JWT + Redis JTI Blacklist

Read the full contents of:

```
- kebos-backend/app/auth/services.py
- kebos-backend/app/auth/router.py
- kebos-backend/app/core/config.py
- kebos-backend/app/core/dependencies.py
- kebos-backend/requirements.txt
```

Then make ALL of the following changes:

```
1. Add to requirements: cryptography>=41.0.0

2. Create kebos-backend/app/core/rsa_keys.py:
    - load_or_generate_keypair() that checks RSA_PRIVATE_KEY and RSA_PUBLIC_KEY env
      vars
    - In production (ENVIRONMENT=production): raises RuntimeError if keys missing
    - In development: generates RSA-4096 keypair with ephemeral WARNING
    - get_private_key() and get_public_key() accessors

3. In auth/services.py:
    - Replace algorithm="HS256" with algorithm="RS256"
    - Use RSA private key from rsa_keys.get_private_key()
    - Add jti=str(uuid.uuid4()) claim
    - Set expiry to 15 minutes (ACCESS_TOKEN_EXPIRE_MINUTES=15)
```

4. In `dependencies.py`:
 - Replace HS256 verification with RS256 using RSA public key
 - Extract `jti` claim after decoding
 - Check `redis.get(f"jti:{tenant_id}:{jti}")` – if exists, raise HTTP 401 "Token has been revoked"
 - Note: `tenant_id` namespaced blacklist keys for multi-tenancy readiness
 5. In `auth/router.py` logout endpoint:
 - `redis_client.setex(f"jti:{tenant_id}:{jti}", token_ttl_seconds, "1")`
 - Delete the cookie: `response.delete_cookie("access_token")`
 6. Use existing Redis connection from Celery/cache config – do not create new connection.
- Show complete diff. Flag every place assuming HS256 for manual review.

Prompt 1.3 — TOTP MFA

Read the full contents of:

- `kebos-backend/app/auth/models.py`
- `kebos-backend/app/auth/router.py`
- `kebos-backend/app/auth/services.py`
- `kebos-backend/alembic/versions/` (most recent migration)

Add TOTP Multi-Factor Authentication:

1. Add to requirements: `pyotp>=2.9.0` `qrcode>=7.4.2`
2. In `auth/models.py`:
 - Add: `totp_secret_encrypted = Column(Text, nullable=True)`
NOTE: field name is `totp_secret_ENCRYPTED` – plaintext is never stored
 - Add: `totp_enabled = Column(Boolean, default=False, server_default='false')`
 - IMPORTANT: `totp_secret` is always encrypted via Vault transit engine before writing to this column. Never store the raw pyotp secret.
3. Alembic migration: `add_totp_to_users`
 - Add `totp_secret_encrypted` (TEXT, nullable)
 - Add `totp_enabled` (BOOLEAN, default false)
4. In `auth/services.py`, create:

```
async def encrypt_totp_secret(secret: str) -> str:
    """Encrypt TOTP secret via Vault transit before DB storage."""
    return await vault_client.encrypt("transit/encrypt/kebos-totp", secret)

async def decrypt_totp_secret(encrypted: str) -> str:
    """Decrypt TOTP secret from DB via Vault transit for verification."""
    return await vault_client.decrypt("transit/decrypt/kebos-totp", encrypted)
```

5. In `auth/router.py`, add three endpoints:

```
POST /auth/mfa/setup
- Generate pyotp.random_base32() secret
- Encrypt via encrypt_totp_secret() BEFORE storing temporarily
- Return: {"secret": secret, "qr_uri": provisioning_uri}

POST /auth/mfa/verify-setup
- Body: {"code": "123456"}
- Decrypt temp secret, validate code
- If valid: encrypt_totp_secret() then save to totp_secret_encrypted
- Set totp_enabled=True

POST /auth/mfa/verify
- Body: {"code": "123456"}
- Decrypt totp_secret_encrypted for verification only
- pyotp.TOTP(decrypted_secret).verify(code, valid_window=1)
- If valid: set session cookie
```

6. Login flow modification:

- `totp_enabled=False` → set cookie (existing flow)
- `totp_enabled=True` → return {"status": "mfa_required"} + short-lived challenge cookie

Show complete diff. TOTP secret must never appear in logs.

Prompt 1.3b [NEW v3] — FIDO2 Hardware Key + Session Risk Scoring

Read the full contents of:

- `kebos-backend/app/auth/router.py`
- `kebos-backend/app/auth/models.py`
- `kebos-backend/app/core/config.py`
- `kebos-backend/app/auth/services.py`

Add FIDO2/WebAuthn support and continuous session risk scoring.

1. Add to requirements: webauthn>=1.9.0 geoip2>=4.6.0

2. In auth/models.py:

- Add: fido2_credentials = Column(JSONB, nullable=True, default=list)
Stores list of registered credential IDs per user
- Add: last_known_ip = Column(String(45), nullable=True)
- Add: last_known_location = Column(String(100), nullable=True)
- Add: last_seen_at = Column(DateTime, nullable=True)
- Add: device_fingerprints = Column(JSONB, nullable=True, default=list)

3. In auth/router.py, add FIDO2 endpoints:

POST /auth/fido2/register/begin

- Requires authenticated session
- Returns WebAuthn registration options

POST /auth/fido2/register/complete

- Body: registration response from browser
- Stores credential in user.fido2_credentials

POST /auth/fido2/authenticate/begin

- Returns WebAuthn authentication options for user

POST /auth/fido2/authenticate/complete

- Body: authentication response from browser
- If valid: set session cookie

4. Create kebos-backend/app/auth/session_risk.py:

```
import math
```

```
from datetime import datetime, timezone
```

```
from geoip2 import database as geoip_db
```

```
class SessionRiskScorer:
```

```
    def __init__(self, geoip_db_path: str):  
        self.reader = geoip_db.Reader(geoip_db_path)
```

```
    def score(self, user, current_ip: str, current_va: str) -> float:  
        """Returns 0.0 (no risk) to 1.0 (block immediately)."""  
        signals = []
```

```

# Impossible travel detection
if user.last_known_ip and user.last_known_ip != current_ip:
    distance_km = self._haversine_distance(
        user.last_known_location, current_ip)
    time_diff_h = max(
        (datetime.now(timezone.utc) -
user.last_seen_at).total_seconds() / 3600,
        0.01)
    speed_kmh = distance_km / time_diff_h
    if speed_kmh > 900: # physically impossible
        signals.append(1.0) # auto-block
    elif speed_kmh > 400:
        signals.append(0.7)

# Off-hours access (outside 6am-10pm IST)
ist_hour = (datetime.now(timezone.utc).hour + 5.5) % 24
if ist_hour < 6 or ist_hour > 22:
    signals.append(0.3)

# New device fingerprint
fp = self._fingerprint(current_ip, current_ua)
if fp not in (user.device_fingerprints or []):
    signals.append(0.4)

return max(signals) if signals else 0.0

def _haversine_distance(self, prev_location: str, current_ip: str) ->
float:
    try:
        record = self.reader.city(current_ip)
        # Parse prev_location as "lat,lon"
        if prev_location:
            lat1, lon1 = map(float, prev_location.split(','))
            lat2 = record.location.latitude or 0
            lon2 = record.location.longitude or 0
            R = 6371
            dlat = math.radians(lat2 - lat1)
            dlon = math.radians(lon2 - lon1)
            a = math.sin(dlat/2)**2 + math.cos(math.radians(lat1)) * \
                math.cos(math.radians(lat2)) * math.sin(dlon/2)**2
            return R * 2 * math.asin(math.sqrt(a))
    except Exception:
        pass
    return 0.0

```

```
def _fingerprint(self, ip: str, ua: str) -> str:
    import hashlib
    return hashlib.sha256(f"{ip}:{ua}".encode()).hexdigest()[:16]
```

5. In `dependencies.py`, on every authenticated request:

- Call `SessionRiskScorer.score(user, request.client.host, request.headers.get("User-Agent"))`
- If `score >= 1.0`: revoke session, return HTTP 401 "Session terminated: impossible travel"
- If `score >= 0.6`: return HTTP 403 "Step-up authentication required" (frontend must re-challenge with TOTP or FIDO2)
- If `score >= 0.3`: log warning to audit trail but allow request

6. In `config.py`, add:

```
AUTH_REQUIRE_FIDO2_FOR_GOVERNMENT: bool = True
TENANT_AUTH_POLICIES: dict = {
    "government": {"require_fido2": True, "allow_totp": False,
"session_minutes": 60},
    "bfsi": {"require_fido2": False, "allow_totp": True, "session_minutes":
240},
    "enterprise": {"require_fido2": False, "allow_totp": True,
"session_minutes": 480},
}
```

Show complete diff.

PHASE 2 — QMIND INTEGRATION

Prompt 2.1 — QMind Microservice Docker Setup

Read the full contents of:

- `docker-compose.yml` at the project root
- `qmind_enterprise/api/enterprise_api.py`
- `qmind_enterprise/requirements.txt`
- `kebos-backend/app/core/config.py`

Then make ALL of the following changes:

1. In `docker-compose.yml`, add the `qmind` service on port 8001.

NOTE: This prompt sets up the basic service.

Network segmentation is added in Phase 5 (Prompt 5.1).

2. Create qmind_enterprise/start.sh:

```
#!/bin/sh
set -e
python -m kafka_consumer &
CONSUMER_PID=$!
uvicorn api.enterprise_api:app --host 0.0.0.0 --port 8001 &
API_PID=$!
wait -n $CONSUMER_PID $API_PID
```

3. Create qmind_enterprise/Dockerfile if it does not exist:

```
FROM python:3.12-slim
WORKDIR /app
RUN groupadd -r qmind && useradd -r -g qmind qmind
RUN apt-get update && apt-get install -y cmake gcc g++ libssl-dev && \
    rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN pip install liboqs-python --no-cache-dir
COPY --chown=qmind:qmind . .
RUN chmod +x start.sh
USER qmind
ENV USE_REAL_PQC=true
CMD ["/start.sh"]
NOTE: Non-root user (qmind) required — no-new-privileges enforced in compose.
```

4. In enterprise_api.py, ensure GET /health returns:

```
{"status": "ok", "pqc": bool, "feeds_loaded": int, "uptime_seconds": float}
```

5. In config.py add: QMIND_URL, QMIND_TIMEOUT_SECONDS=5

Show complete diff.

Prompt 2.1b — /analyze and /signals/inject endpoints

Read the full contents of:

- qmind_enterprise/api/enterprise_api.py
- qmind_enterprise/signal_engine/ (ALL files)
- qmind_enterprise/external_dataset_loader.py

Make ALL of the following changes:

1. POST /analyze with exact Pydantic request/response schemas.
Response MUST always include all 10 category_scores keys:
Phishing, Malware, C2_Infrastructure, Botnet_IP, Credential_Leak,
Supply_Chain, Insider_Threat, DDoS, CVE_Exploitation, Benign
Plus: lead_category, confidence, reversibility, advisory,
manipulability_score, processing_time_ms, analysed_at

IRREVERSIBLE_CATEGORIES = {C2_Infrastructure, Botnet_IP, Supply_Chain,
CVE_Exploitation}
reversibility = "IRREVERSIBLE" if lead_category in IRREVERSIBLE_CATEGORIES
and confidence >= 0.75 else "REVERSIBLE"

LLM-PROMPT FIX from v2.0: _parse_report_sections() must use JSON-mode.
The /analyze response itself must return structured JSON – never rely on
line-number prefix parsing of LLM output anywhere in QMind.
2. POST /signals/inject with SignalInjectRequest / SignalInjectResponse schemas.
Source field must accept: "honeypot" | "analyst" | "external" | "ct_log" |
"paste_monitor" | "apk_monitor" | "domain_monitor" | "crawler"
Log all injections to audit trail with Dilithium-3 signature.
3. GET /health returns full schema.

Show complete diff with side-by-side comparison of old vs new /analyze schema.

Prompt 2.2 — Kafka Bridge: CatBoost → QMind

Read the full contents of:

- kebos-backend/app/threat_detection/ (ALL files)
- kebos-backend/app/core/config.py
- qmind_enterprise/api/enterprise_api.py

Create ThreatIndicatorPublisher in kafka_publisher.py using aiokafka.

After every CatBoost prediction, publish to threat.indicators topic.

Mark every addition with # KAFKA_PUBLISH_ADDED.

Add to requirements: aiokafka>=0.10.0

Show complete diff.

Prompt 2.2-patch — Fix async/sync mismatch in kafka_consumer.py

Read `qmind_enterprise/kafka_consumer.py`.

If it uses `kafka-python`'s synchronous `KafkaConsumer`, rewrite completely using `AIOKafkaConsumer`. The synchronous consumer blocks `asyncio` permanently.

Full rewrite using `aiokafka` with:

- `get_producer()` with `acks="all"`
- `publish_qmind_result()` publishing to `qmind.results`
- `consume_threat_indicators()` with `AIOKafkaConsumer`, `group_id="qmind-threat-consumer"`
- Error handling per message (never crash the consumer loop)

CRITICAL v3.0 ADDITION: Add `done_callback` to the `asyncio` task in `start.sh`:

In the process launcher, capture any consumer crash and log to `stderr` with full traceback before the process exits. Silent crashes are the #1 silent failure.

Remove `kafka-python` from requirements. Add `aiokafka>=0.10.0`.

Show complete diff.

Prompt 2.2b — QMind Results Consumer on Kebos Backend

Read the full contents of:

- `kebos-backend/app/threat_detection/` (ALL files)
- `kebos-backend/app/core/config.py`
- `kebos-backend/app/main.py`
- `kebos-backend/alembic/versions/` (most recent migration)

CRITICAL GAP: QMind publishes to `qmind.results` but Kebos never consumes them.

1. Create `qmind_result_consumer.py` consuming `qmind.results` topic.
`group_id="kebos-qmind-results-consumer"`
2. Create `update_threat_with_qmind_result()` in `services.py`:
 - Update `ThreatEvent` with all QMind fields
 - `confidence >= 0.75` → `status = "CONFIRMED_THREAT"`
 - `confidence < 0.75` → `status = "SUSPICIOUS"`
 - Push `WebSocket` event to dashboard
 - If `CONFIRMED_THREAT`: trigger honeypot deploy (Phase 4) and `analyst.feedback` topic publish (Phase 9 – stub now, implement in 9.1)

```

3. Alembic migration: add_qmind_columns_to_threat_events
    Add: qmind_lead_category, qmind_confidence, qmind_category_scores (JSONB),
        qmind_reversibility, qmind_advisory, qmind_analysed_at

4. In main.py startup:
    task = asyncio.create_task(consume_qmind_results(), name="qmind-results-
consumer")
    task.add_done_callback(lambda t: logger.error(
        f"QMind results consumer crashed: {t.exception()}", exc_info=t.exception())
    )

```

Show complete diff. Show existing ThreatEvent model fields before making changes.

Prompt 2.3 — Replace Fake PQC with Real liboqs

Read the full contents of:

- kebos-backend/app/messaging/crypto_pq.py
- qmind_enterprise/crypto/ (ALL files)
- qmind_enterprise/pqc_signatures.py

CRITICAL: crypto_pq.py uses fake polynomial approximations. Replace with real liboqs.

1. Add to requirements: liboqs-python>=0.9.0

2. Rewrite crypto_pq.py:

- Kyber768Encapsulator: generate_keypair, encapsulate, decapsulate using oqs.KeyEncapsulation
- Dilithium3Signer: generate_keypair, sign, verify using oqs.Signature
- HYBRID ENCRYPTION: Kyber-768 + AES-256-GCM (not Kyber alone)


```

shared_secret = kyber.decapsulate(sk, ciphertext)
aes_key = HKDF(shared_secret, length=32, salt=None, info=b"kebos-hybrid-v1")
encrypted = AES_GCM.encrypt(aes_key, plaintext)
            
```

This gives both classical and quantum resistance.
- Module-level oqs import check with _REAL_PQC flag
- Warning if _REAL_PQC=False – never silently proceed

3. Audit log signing: SHA-256 hash → Dilithium3Signer.sign() → store hex sig + key ref

4. Add `USE_REAL_PQC=true` to `.env.example`

Show complete diff. List every incompatibility with existing callers explicitly.

Prompt 2.4 — QMind → LLM → SOC Report Pipeline

Read the full contents of:

- `kebos-backend/app/genai_assistant/` (ALL files)
- `qmind_enterprise/api/enterprise_api.py`
- `kebos-backend/app/threat_detection/` (ALL files)
- `kebos-backend/app/audit_logger/` (ALL files)

Create `SOCReportGenerator` with tiered LLM routing.

1. Create `soc_report_generator.py`:

```
class LLMDDataSanitiser:
    SAFE_FOR_EXTERNAL = {
        'lead_category', 'confidence', 'category_scores',
        'reversibility', 'advisory', 'indicator_type',
        'threat_level', 'recommended_actions'
    }
    NEVER_EXTERNAL = {
        'source_ip', 'internal_hostname', 'username',
        'customer_id', 'asset_inventory', 'raw_logs',
        'tenant_id', 'employee_id', 'account_number'
    }
    def sanitise(self, payload: dict, classification: str) -> dict:
        if classification in ('CONFIDENTIAL', 'RESTRICTED'):
            raise ValueError(f"Use local Gemma for {classification} data")
        return {k: v for k, v in payload.items()
                if k in self.SAFE_FOR_EXTERNAL}

class SOCReportGenerator:
    def __init__(self):
        self.llm_groq = None      # external - PUBLIC/INTERNAL data only
        self.llm_gemma = None     # local - CONFIDENTIAL data
        self.sanitiser = LLMDDataSanitiser()

    def _get_llm_for_classification(self, classification: str, tenant_type:
str):
```

```

        if tenant_type == 'government':
            if self.llm_gemma is None:
                raise RuntimeError("Local Gemma required for government
tenants")

            return self.llm_gemma
        if classification in ('CONFIDENTIAL', 'RESTRICTED'):
            if self.llm_gemma is None:
                raise RuntimeError("Local Gemma not wired for confidential
data")

            return self.llm_gemma
        if self.llm_groq is None:
            raise RuntimeError("LLM client not wired. Call wire_llm_clients()
at startup.")
        return self.llm_groq

```

PROMPT FIX (CRITICAL): Use JSON-mode system prompt. Never use line-number prefix parsing.

System prompt: "Respond ONLY with valid JSON. No preamble, no markdown, no code blocks."

The prompt must request a JSON object with keys:

incident_summary, technical_details, threat_context, recommended_actions, cert_in_fields, spl_hunt_query, kql_hunt_query

Output validation – check for prompt injection before returning:

```

INJECTION_PATTERNS = ['ignore previous', 'system prompt', 'you are now',
                      '<script>', '{{', '%', 'eval(', 'exec(']

```

Raise SecurityException if any pattern found in raw LLM response.

2. POST /api/v1/soc/generate-report endpoint.

Returns HTTP 503 if RuntimeError raised.

Returns HTTP 400 if SecurityException raised.

3. Frontend SOC assistant page: input, submit, section cards, SPL/KQL code blocks with copy.

Show complete diff.

Prompt 2.4-patch — Wire SOCReportGenerator LLM Clients at Startup

Read the full contents of:

- kebos-backend/app/genai_assistant/ (ALL files)

- kebos-backend/app/genai_assistant/soc_report_generator.py
- kebos-backend/app/main.py

Wire BOTH Groq and local Gemma clients at startup.

1. In main.py startup handler:

```
from app.genai_assistant.soc_report_generator import wire_llm_clients
from app.genai_assistant.<module> import get_groq_client, get_gemma_client

@app.on_event("startup")
async def start_background_tasks():
    # QMind consumer
    task = asyncio.create_task(consume_qmind_results(), name="qmind-results-
consumer")
    task.add_done_callback(handle_consumer_crash)
    # LLM clients
    groq_client = await get_groq_client()
    gemma_client = await get_gemma_client()
    wire_llm_clients(groq=groq_client, gemma=gemma_client)
    logger.info("SOCReportGenerator: Both LLM clients wired successfully")
    # Crawler startup (Phase 6 – stub here, implement in 6.1)
    # asyncio.create_task(start_ct_log_monitor())
```

2. wire_llm_clients() must be idempotent.

3. Log "SOCReportGenerator: Both LLM clients wired successfully" at startup.

Show complete diff.

PHASE 3 — API GATEWAY + RATE LIMITING

Prompt 3.1 — Rate Limiting + Security Middleware

Read the full contents of:

- kebos-backend/app/main.py
- kebos-backend/app/auth/router.py
- kebos-backend/requirements.txt
- Every router.py file in each module

Make ALL of the following changes:

1. Add to requirements: slowapi>=0.1.9 limits>=3.6.0

2. CRITICAL v3.0 FIX: Use Redis backend for slowapi – NOT in-memory.
In-memory rate limits are per-process and bypass in multi-replica deployments.

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from limits.storage import RedisStorage

limiter = Limiter(
    key_func=get_remote_address,
    storage_uri=settings.REDIS_URL,    # Redis backend – required
    strategy="fixed-window"
)
app.state.limiter = limiter
```

3. Rate limits:
 - POST /auth/login: @limiter.limit("10/minute"), @limiter.limit("100/day")
 - POST /analyze, POST /enrich, POST /soc/generate-report:
@limiter.limit("30/minute")
 - All other endpoints: @limiter.limit("100/minute")

4. CORS: Replace allow_headers=["*"] with explicit list.
Replace allow_origins=["*"] with settings.ALLOWED_ORIGINS.

5. X-Request-ID middleware.

Show complete diff. List every endpoint where limiter was added.

Prompt 3.1b [NEW v3] — Rate Limiting: Redis Backend Verification + Syslog TCP+TLS

Read the full contents of:

- kebos-backend/app/main.py
- kebos-backend/app/core/config.py
- kebos-backend/app/audit_logger/ (ALL files)

Two critical fixes that close the remaining ZTA observability gaps.

1. Verify slowapi Redis backend is correctly configured:
 - In main.py startup, after limiter is created:

```
from limits.storage import RedisStorage
assert isinstance(limiter._storage, RedisStorage), \
```

```
"CRITICAL: slowapi is not using Redis backend. Rate limits are per-
process only."
```

```
logger.info(f"Rate limiter using Redis at: {settings.REDIS_URL}")
```

2. Fix syslog to use TCP+TLS (NEVER UDP):

Create kebos-backend/app/audit_logger/tls_syslog_handler.py:

```
import ssl, socket, logging
```

```
class TLSSyslogHandler(logging.Handler):
```

```
    def __init__(self, host: str, port: int = 6514, ca_cert: str = None):
```

```
        super().__init__()
```

```
        self.host = host
```

```
        self.port = port
```

```
        self.ctx = ssl.create_default_context()
```

```
        if ca_cert:
```

```
            self.ctx.load_verify_locations(ca_cert)
```

```
    def emit(self, record):
```

```
        try:
```

```
            msg = self.format(record) + "\n"
```

```
            with socket.create_connection((self.host, self.port), timeout=5) as
```

```
sock:
```

```
                with self.ctx.wrap_socket(sock, server_hostname=self.host) as
```

```
ssock:
```

```
                    ssock.sendall(msg.encode("utf-8"))
```

```
        except Exception:
```

```
            self.handleError(record)
```

Replace any existing SysLogHandler(address=(..., 514)) with TLSSyslogHandler.

CERT-In audit trail validity requires TCP+TLS – UDP has no integrity guarantee.

3. Add to config.py:

```
SYSLOG_HOST: str = ""
```

```
SYSLOG_PORT: int = 6514
```

```
SYSLOG_CA_CERT: str = "" # path to CA cert for TLS verification
```

Show complete diff.

Prompt 3.2 — CERT-In Report Generator

Read the full contents of:

- kebos-backend/app/audit_logger/ (ALL files)
- kebos-backend/app/genai_assistant/soc_report_generator.py
- qmind_enterprise/pqc_signatures.py

1. Add to requirements: reportlab>=4.0.0 jinja2>=3.1.0

2. Create cert_in_template.html.j2 with all required CERT-In fields.

3. Create cert_in_generator.py:

```
class CERTInReportGenerator:
    def generate_report(self, incident_data, soc_sections, signing_key_path) ->
bytes:
    # 1. Render Jinja2 -> HTML
    # 2. Convert to PDF via ReportLab
    # 3. SHA-256 hash of PDF bytes
    # 4. Dilithium3Signer.sign() the hash
    # 5. Embed X-Dilithium3-Signature + X-Dilithium3-PubKey in PDF metadata
    # 6. Return PDF bytes
```

TIMING REQUIREMENT: Total generation time must be logged.

If generation takes > 300 seconds, log a WARNING – 6-hour window is at risk.

4. POST /api/v1/reports/cert-in endpoint.

Flow: QMind /analyze -> SOC report -> CERT-In PDF -> Dilithium sign -> return

5. Register reporting router in main.py.

Show complete diff.

PHASE 4 — DECEPTION GRID + DIGITALTWIN + SIEM

Prompt 4.1 — HoneyGrid Manager

Read the full contents of:

- kebos-backend/app/threat_detection/ (ALL files)
- kebos-backend/app/main.py
- docker-compose.yml
- qmind_enterprise/api/enterprise_api.py

Goal: deploy automated honeypots when QMind detects CONFIRMED_THREAT.

1. Add to requirements: docker>=6.1.0
2. Create honeygrid.py with HoneyGridManager class.

CRITICAL v3.0 FIX: Do NOT mount /var/run/docker.sock directly.

Use Tecnativa docker-socket-proxy:

- docker-proxy service in docker-compose.yml restricts to CONTAINERS API only
- HoneyGridManager connects to tcp://docker-proxy:2375 NOT the raw socket

docker-compose addition:

docker-proxy:

image: tecnativa/docker-socket-proxy

environment:

CONTAINERS: 1

NETWORKS: 1

POST: 1

IMAGES: 1

VOLUMES: 0 # deny volume API

AUTH: 0 # deny auth API

SECRETS: 0 # deny secrets API

volumes:

- /var/run/docker.sock:/var/run/docker.sock:ro

networks:

- app-net

HoneyGridManager connects via:

docker.DockerClient(base_url="tcp://docker-proxy:2375")

3. honeypot_images = {
 'ssh': 'cowrie/cowrie:latest',
 'http_api': 'traefik/whoami:latest',
 'database': 'postgres:16-alpine',
 'upi_endpoint': 'traefik/whoami:latest', # BFSI-specific deception
}
4. iptables DNAT: validate attacker_ip with ipaddress.ip_address() before any subprocess call. Never interpolate unsanitised input.
5. Deception router: POST /api/deception/events, GET /api/deception/active, DELETE /api/deception/{attacker_ip}

```
6. POST /api/deception/events calls http://qmind:8001/signals/inject with
source="honeypot"
```

```
7. In docker-compose.yml: kebos_deception_net with internal: true
```

```
Show complete diff.
```

Prompt 4.1b [NEW v3] — Honeytoken Deployment

Read the full contents of:

- kebos-backend/app/deception/honeygrid.py
- kebos-backend/app/deception/router.py
- kebos-backend/app/audit_logger/ (ALL files)

Add honeytoken support – fake credentials that trigger zero-false-positive breach alerts.

1. Create kebos-backend/app/deception/honeytokens.py:

```
from enum import Enum
import secrets, hashlib, hmac

class HoneytokenType(str, Enum):
    AWS_ACCESS_KEY    = "aws_access_key"
    DB_PASSWORD       = "db_password"
    API_KEY           = "api_key"
    JWT_TOKEN         = "jwt_token"
    UPI_CREDENTIAL    = "upi_credential"    # BFSI-specific
    SWIFT_TOKEN       = "swift_token"       # BFSI-specific

class HoneytokenManager:
    def generate(
        self,
        token_type: HoneytokenType,
        tenant_id: str,
        description: str
    ) -> dict:
        """
        Generate a fake credential that looks real but triggers alert on use.
        Returns: {token_id, value, type, tenant_id, created_at}
        The token_id is stored in DB. The value is deployed into customer
```

```

environment.
    """
    token_id = secrets.token_hex(16)
    value = self._generate_realistic_value(token_type)
    # Store hash of value (not value itself) in DB
    value_hash = hashlib.sha256(value.encode()).hexdigest()
    return {"token_id": token_id, "value": value, "hash": value_hash}

def _generate_realistic_value(self, token_type: HoneytokenType) -> str:
    if token_type == HoneytokenType.AWS_ACCESS_KEY:
        return
        f"AKIA{''.join(secrets.choice('ABCDEFGHIJKLMNOPQRSTUVWXYZ234567') for _ in
range(16))}"
    elif token_type == HoneytokenType.DB_PASSWORD:
        return f"honey_{secrets.token_urlsafe(24)}_db"
    elif token_type == HoneytokenType.API_KEY:
        return f"sk-honey-{secrets.token_urlsafe(32)}"
    elif token_type == HoneytokenType.UPI_CREDENTIAL:
        return f"upi_honey_{secrets.token_hex(12)}@kebos"
    else:
        return secrets.token_urlsafe(48)

async def alert_on_use(
    self,
    token_value: str,
    source_ip: str,
    tenant_id: str
):
    """
    Called when a honeytoken is used. Confidence = 1.0 - zero false
positives.
    Inject directly into QMind with maximum confidence.
    """
    import httpx
    payload = {
        "indicator_value": source_ip,
        "indicator_type": "ip",
        "strength": 1.0,
        "confidence": 1.0,
        "source": "honeypot",
        "metadata": {
            "trigger": "honeytoken_used",
            "token_type": "credential",
            "tenant_id": tenant_id

```

```
        }
    }
    async with httpx.AsyncClient() as client:
        await client.post("http://qmind:8001/signals/inject", json=payload,
            timeout=5.0)
```

2. Create honeypot model in deception/models.py:

```
- id, token_type, value_hash, tenant_id, description, created_at,
last_triggered_at
- Alembic migration: create_honeypots_table
```

3. Add endpoints to deception/router.py:

```
POST /api/deception/honeypots          - generate honeypot (ADMIN only)
GET  /api/deception/honeypots          - list all honeypots (ADMIN only)
POST /api/deception/honeypots/trigger - called when honeypot is used
    Body: {"token_value": str, "source_ip": str}
    Action: verify token_value hash matches DB record → alert_on_use()
```

4. Internal honeypots for Kebos itself (not customer-facing):

In main.py startup, register a check:

```
INTERNAL_HONEYPOTS = {
    "AKIAFAKE123456KEBOSHONEY": "aws_key_in_env",
    "honeypostgres_kebos_internal_DO_NOT_USE": "db_password",
}
```

If any request contains an internal honeypot value in headers/body,
log CRITICAL breach alert immediately.

Show complete diff.

Prompt 4.2 — Digital Twin Dry-Run Mitigation

Read the full contents of:

```
- kebos-backend/app/threat_detection/ (ALL files)
- qmind_enterprise/api/enterprise_api.py
```

Create DigitalTwinSimulator with full simulate_action() implementation.

SimulationResult dataclass with:

```
action_type, target, total_events_in_window, estimated_false_positives,
impact_score, safe_to_execute (True if impact_score < 0.05), simulated_at,
recommendation
```

```
simulate_action():
1. Pull last replay_window_minutes of events from TimescaleDB for target
2. Apply proposed rule to each historical event
3. Count events that would have been false positives
4. impact_score = estimated_false_positives / max(total_events_in_window, 1)
5. safe_to_execute = impact_score < 0.05
6. recommendation = "SAFE TO EXECUTE" or "REVIEW REQUIRED: {N} potential FPs in {window}min"
```

REVERSIBLE actions: execute immediately + notify analyst

IRREVERSIBLE actions: Digital Twin dry-run → if not safe:

create_analyst_review_task

→ analyst approval required → execute

POST /api/v1/simulation/dry-run endpoint.

Show complete diff.

Prompt 4.3 — SIEM Integration Module

Read the full contents of:

- kebos-backend/app/siem_integration/ (ALL files)
- kebos-backend/app/core/config.py
- qmind_enterprise/api/enterprise_api.py

Create four integration modes:

1. SplunkHECForwarder: POST to {hec_url}/services/collector
Include all 10 category_scores, dilithium_signature, reversibility, advisory
2. CEFSyslogForwarder:
CEF:0|Pynevera|Kebos|1.0|{category}|{description}|{severity}|...
CRITICAL v3.0: Use TLSSyslogHandler (from 3.1b) NOT raw UDP syslog
Import from audit_logger.tls_syslog_handler
3. POST /api/v1/enrich endpoint:
 - Redis cache 15 minutes (key: f"enrich:{tenant_id}:{indicator}")
NOTE: tenant_id namespaced cache keys for multi-tenancy
 - POST to QMind /analyze with timeout=5s
 - @limiter.limit("30/minute")

4. STIXTAXIIServer:
 - generate_stix_bundle(): STIX 2.1 Indicator objects from confirmed threats
 - serve_taxii_collection(): TAXII 2.1 collection manifest
 - Include Dilithium-3 signature in STIX custom property: x_kebos_pqc_signature
5. Register integrations router in main.py.

Show complete diff.

PHASE 5 — ZERO TRUST ARCHITECTURE [ALL NEW v3]

Prompt 5.1 [NEW v3] — Docker Network Micro-segmentation

Read the full contents of:

- docker-compose.yml
- kebos-backend/app/core/config.py

CURRENT STATE: All services are on a single Docker bridge network.

A compromised container can freely reach PostgreSQL, Redis, Kafka, and QMind. This is the highest-severity architectural gap in the current deployment.

Make ALL of the following changes:

1. In docker-compose.yml, replace the single kebos-network with 3 isolated networks:

```
networks:
  frontend-net:
    driver: bridge
    # Only: nginx/Cloudflare proxy → Kebos FastAPI
  app-net:
    driver: bridge
    # Only: Kebos FastAPI ↔ QMind ↔ Redis ↔ docker-proxy
  data-net:
    driver: bridge
    # Only: Kebos FastAPI ↔ PostgreSQL ↔ Kafka
```

2. Assign networks per service (strict – no service gets more networks than it needs):

```
kebos-backend:  [frontend-net, app-net, data-net]
```

```
qmind:          [app-net]          # QMind talks to Kebos only via Kafka/HTTP
postgres:       [data-net]         # DB not reachable from frontend-net
kafka:          [data-net]         # Kafka not reachable from frontend-net or
app-net directly
redis:          [app-net]          # Redis not reachable from frontend-net or
data-net
docker-proxy:   [app-net]          # Docker proxy only on app-net
kebos_deception_net:
  driver: bridge
  internal: true # honeypot containers have no internet access
```

3. In config.py, add:

```
DOCKER_PROXY_URL: str = "tcp://docker-proxy:2375" # never raw socket
```

4. Verify existing service-to-service URLs match new network topology:

- qmind URL: http://qmind:8001 ✓ (both on app-net)
- kafka URL: kafka:9092 ✓ (kebos on data-net, kafka on data-net)
- redis URL: redis://redis:6379 ✓ (both on app-net)
- postgres URL: postgresql://postgres:5432 ✓ (both on data-net)

Show complete diff of docker-compose.yml network configuration.

Prompt 5.2 [NEW v3] — Vault PKI: Service mTLS Certificates

Read the full contents of:

- kebos-backend/app/core/config.py
- kebos-backend/app/main.py
- qmind_enterprise/api/enterprise_api.py

Add mutual TLS between Kebos backend and QMind using Vault PKI engine.

1. Create kebos-backend/app/core/vault_pki.py:

```
import hvac, ssl, tempfile, os
from pathlib import Path

class VaultPKIClient:
    def __init__(self, vault_addr: str, vault_token: str):
        self.client = hvac.Client(url=vault_addr, token=vault_token)

    async def issue_service_cert(
```

```

        self,
        service_name: str,
        ttl: str = "48h"
    ) -> tuple[str, str, str]:
        """Issues a TLS cert for a service. Returns (cert_pem, key_pem,
ca_pem)."""
        result = self.client.secrets.pki.generate_certificate(
            mount_point="pki",
            name="services",
            common_name=f"{service_name}.internal",
            ttl=ttl,
            extra_params={"ip_sans": "127.0.0.1"}
        )
        data = result["data"]
        return data["certificate"], data["private_key"], data["issuing_ca"]

    async def write_certs_to_temp(self, service_name: str) -> dict:
        """Writes certs to temp files for uvicorn ssl_keyfile/ssl_certfile."""
        cert, key, ca = await self.issue_service_cert(service_name)
        tmpdir = tempfile.mkdtemp(prefix="kebos-certs-")
        paths = {
            "cert": f"{tmpdir}/cert.pem",
            "key": f"{tmpdir}/key.pem",
            "ca": f"{tmpdir}/ca.pem",
        }
        Path(paths["cert"]).write_text(cert)
        Path(paths["key"]).write_text(key)
        Path(paths["ca"]).write_text(ca)
        return paths

```

2. Create kebos-backend/app/core/mtlsmiddleware.py:

```

from starlette.middleware.base import BaseHTTPMiddleware

class MTLSMiddleware(BaseHTTPMiddleware):
    """Verifies that inbound service-to-service requests present a valid client
cert."""
    EXEMPT_PATHS = {"/health", "/auth/login", "/auth/me"}

    async def dispatch(self, request, call_next):
        if request.url.path in self.EXEMPT_PATHS:
            return await call_next(request)
        # In production, uvicorn with ssl_ca_certs="require" enforces mTLS at
        TLS layer

```



```
# This middleware validates the cert subject for service identity
client_cert = request.scope.get("ssl_object")
if client_cert:
    peer_cert = client_cert.getpeercert()
    subject = dict(x[0] for x in peer_cert.get("subject", []))
    request.state.service_id = subject.get("commonName", "unknown")
return await call_next(request)
```

3. In main.py startup:

- If ENVIRONMENT == "production":
pki = VaultPKIClient(settings.VAULT_ADDR, settings.VAULT_TOKEN)
cert_paths = await pki.write_certs_to_temp("kebos-backend")
logger.info(f"mTLS certs issued for kebos-backend, valid 48h")
- Register MTLSMiddleware

4. In config.py add:

```
VAULT_PKI_ENABLED: bool = False    # True in production
VAULT_PKI_MOUNT: str = "pki"
SERVICE_CERT_TTL: str = "48h"
SERVICE_CERT_RENEWAL_AT: str = "24h" # renew when 50% of TTL elapsed
```

Show complete diff.

Prompt 5.3 [NEW v3] — PostgreSQL Row-Level Security (Multi-tenancy Foundation)

Read the full contents of:

- kebos-backend/app/auth/models.py
- kebos-backend/app/threat_detection/models.py (or equivalent ThreatEvent model)
- kebos-backend/alembic/versions/ (most recent migration)
- kebos-backend/app/core/config.py

Add tenant_id to all tables and enable row-level security.

This is the multi-tenancy foundation – data isolation before the full multi-tenant UI is built in Phase 11.

1. Alembic migration: add_tenant_id_and_rls

Add tenant_id column to ALL tables:

- users: tenant_id UUID NOT NULL DEFAULT '00000000-0000-0000-0000-000000000001'
- threat_events: tenant_id UUID NOT NULL
- audit_logs: tenant_id UUID NOT NULL

- honeytokens: tenant_id UUID NOT NULL (already added in 4.1b)
- cases: tenant_id UUID NOT NULL (will be used in Phase 10)

Create tenants table:

- id UUID PRIMARY KEY DEFAULT gen_random_uuid()
- name VARCHAR(255) NOT NULL UNIQUE
- tenant_type VARCHAR(50) NOT NULL -- 'government', 'bfsi', 'enterprise'
- auth_policy JSONB NOT NULL DEFAULT '{}'
- created_at TIMESTAMP NOT NULL DEFAULT NOW()

Enable row-level security:

```
ALTER TABLE threat_events ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_threat_events ON threat_events
    USING (tenant_id = current_setting('app.current_tenant', TRUE)::uuid);
```

```
ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_audit_logs ON audit_logs
    USING (tenant_id = current_setting('app.current_tenant', TRUE)::uuid);
```

2. Create kebos-backend/app/core/tenant_context.py:

```
from contextlib import asynccontextmanager
from sqlalchemy.ext.asyncio import AsyncSession

@asynccontextmanager
async def tenant_context(db: AsyncSession, tenant_id: str):
    """Sets the PostgreSQL session variable for row-level security."""
    await db.execute(
        f"SET LOCAL app.current_tenant = '{tenant_id}'"
    )
    try:
        yield db
    finally:
        await db.execute("RESET app.current_tenant")
```

3. In dependencies.py, add get_tenant_id():

Extract tenant_id from the authenticated JWT claims.
Every database query must be wrapped in tenant_context().

4. In Redis: namespace all keys with tenant_id:

- JTI blacklist: f"jti:{tenant_id}:{jti}"
- Rate limit: f"rate:{tenant_id}:{ip}:{endpoint}"
- QMind cache: f"enrich:{tenant_id}:{indicator}"

Update all Redis key patterns throughout the codebase.

Show complete diff.

Prompt 5.4 [NEW v3] — Kafka Topic ACLs

Read the full contents of:

- docker-compose.yml
- kebos-backend/app/core/config.py
- qmind_enterprise/kafka_consumer.py

Add Kafka ACLs so each service can only access its own topics.

Currently any service can read/write any topic – a compromised container could inject malicious threat indicators or read intelligence meant for another service.

1. In docker-compose.yml kafka service, enable ACL authorizer:

```
KAFKA_AUTHORIZER_CLASS_NAME: kafka.security.authorizer.AclAuthorizer
KAFKA_ALLOW_EVERYONE_IF_NO_ACL_FOUND: "false"
KAFKA_SUPER_USERS: User:admin
```

2. Create docker/kafka-acls.sh (run once after kafka starts):

```
#!/bin/bash
# Kebos backend: can only WRITE threat.indicators, READ qmind.results
kafka-acls.sh --bootstrap-server kafka:9092 --add \
  --principal User:kebos-backend \
  --operation Write --topic threat.indicators
kafka-acls.sh --bootstrap-server kafka:9092 --add \
  --principal User:kebos-backend \
  --operation Read --topic qmind.results
kafka-acls.sh --bootstrap-server kafka:9092 --add \
  --principal User:kebos-backend \
  --operation Write --topic analyst.feedback

# QMind: can only READ threat.indicators, WRITE qmind.results
kafka-acls.sh --bootstrap-server kafka:9092 --add \
  --principal User:qmind \
  --operation Read --topic threat.indicators
kafka-acls.sh --bootstrap-server kafka:9092 --add \
  --principal User:qmind \
  --operation Write --topic qmind.results
```

```
kafka-acls.sh --bootstrap-server kafka:9092 --add \  
  --principal User:qmind \  
  --operation Read --topic crawler.discoveries
```

```
# HoneyGrid: can only WRITE honeypot.interactions  
kafka-acls.sh --bootstrap-server kafka:9092 --add \  
  --principal User:honeygrid \  
  --operation Write --topic honeypot.interactions
```

```
# Crawler: can only WRITE crawler.discoveries  
kafka-acls.sh --bootstrap-server kafka:9092 --add \  
  --principal User:crawler \  
  --operation Write --topic crawler.discoveries
```

3. Add KAFKA_USERNAME and KAFKA_PASSWORD to config.py for each service.
Each service authenticates to Kafka with its own credentials.

Show complete diff of docker-compose.yml and the ACL script.

Prompt 5.5 [NEW v3] — Egress Control + Supplier Allowlist

Read the full contents of:

- kebos-backend/app/core/config.py
- kebos-backend/app/main.py

Add application-level egress control — every outbound HTTP call must go through a validated allowlist. Unexpected outbound connections are breach indicators.

1. Create kebos-backend/app/core/egress_control.py:

```
import httpx  
from urllib.parse import urlparse
```

```
ALLOWED_EGRESS_DOMAINS = {  
    "api.abuseipdb.com",  
    "feodotracker.abuse.ch",  
    "bazaar.abuse.ch",  
    "services.nvd.nist.gov",  
    "openphish.com",  
    "data.phishtank.com",  
    "urlhaus-api.abuse.ch",
```

```

        "tranco-list.eu",
        "api.groq.com",
        "certstream.calidog.io",
        "ct.googleapis.com",
        "api.cloudflare.com",
    }

    class EgressControlledClient(httpx.AsyncClient):
        """Drop-in replacement for httpx.AsyncClient that enforces egress
        allowlist."""
        async def request(self, method, url, **kwargs):
            parsed = urlparse(str(url))
            if parsed.hostname not in ALLOWED_EGRESS_DOMAINS:
                raise PermissionError(
                    f"Egress blocked: {parsed.hostname} not in allowlist. "
                    f"Add to ALLOWED_EGRESS_DOMAINS if intentional."
                )
            return await super().request(method, url, **kwargs)

```

2. Replace all `httpx.AsyncClient()` usages throughout the codebase with `EgressControlledClient()`. Use `sed` to find all instances:

```
grep -rn "httpx.AsyncClient()" kebos-backend/
```

Replace each one.

3. In `config.py`, add:

```
EGRESS_STRICT_MODE: bool = True # set False only during development
```

If False: log WARNING for each non-allowlisted domain but allow the request

4. Add startup check:

```
logger.info(f"Egress control active. Allowed domains:
{len(ALLOWED_EGRESS_DOMAINS)}")
```

Show complete diff. List every `httpx.AsyncClient()` instance replaced.

PHASE 6 — PROACTIVE INTELLIGENCE + WEB CRAWLING [ALL NEW v3]

Prompt 6.1 [NEW v3] — Certificate Transparency Log Monitor

Read the full contents of:

- `kebos-backend/app/core/config.py`
- `kebos-backend/app/main.py`

- qmind_enterprise/api/enterprise_api.py

Build the CT log monitor – highest value/effort ratio feature in Kebos.
Catches phishing domains at certificate issuance, before the page goes live.

1. Add to requirements: certstream>=1.13 tldextract>=5.1.1

2. Create kebos-backend/app/crawlers/ct_monitor.py:

```
import certstream, asyncio, logging, httpx, tldextract
from app.core.config import settings
from app.core.egress_control import EgressControlledClient

# Patterns for Indian BFSI brands – extend per customer tenant
INDIAN_BRAND_PATTERNS = [
    'sbi', 'hdfc', 'icici', 'axis', 'kotak', 'rbi', 'npci', 'bhim',
    'phonepe', 'paytm', 'googlepay', 'ubi', 'pnb', 'boi', 'canara',
    'indianbank', 'unionbank', 'idbi', 'yesbank', 'indusind',
]

class CTLogMonitor:
    def __init__(self):
        self.logger = logging.getLogger(__name__)
        self._running = False

    def _is_suspicious(self, domain: str) -> bool:
        domain_lower = domain.lower()
        ext = tldextract.extract(domain_lower)
        domain_part = ext.domain + ext.suffix

        for brand in INDIAN_BRAND_PATTERNS:
            if brand in domain_lower:
                # Exclude legitimate domains (e.g., sbi.co.in itself)
                legitimate_tlds = {'.co.in', '.in', '.com', '.net'}
                for tld in legitimate_tlds:
                    if domain_lower == f"{brand}{tld}":
                        return False
                # Domain CONTAINS brand but is NOT the brand itself =
                suspicious

        return True

    return False

    def _ct_callback(self, message, context):
        if message['message_type'] != 'certificate_update':
```

```

        return
domains = message['data']['leaf_cert']['all_domains']
for domain in domains:
    if self._is_suspicious(domain):
        asyncio.run_coroutine_threadsafe(
            self._inject_ioc(domain),
            context['loop']
        )

async def _inject_ioc(self, domain: str):
    payload = {
        "indicator_value": domain,
        "indicator_type": "domain",
        "strength": 0.7,
        "confidence": 0.65,
        "source": "ct_log",
        "metadata": {
            "detection_method": "certificate_transparency",
            "proactive": True
        }
    }
    try:
        async with EgressControlledClient() as client:
            await client.post(
                "http://qmind:8001/signals/inject",
                json=payload,
                timeout=5.0
            )
            self.logger.info(f"CT log IOC injected: {domain}")
    except Exception as e:
        self.logger.error(f"CT log injection failed for {domain}: {e}")

def start(self, loop: asyncio.AbstractEventLoop):
    """Run certstream listener in a background thread."""
    import threading
    self._running = True
    context = {'loop': loop}
    thread = threading.Thread(
        target=certstream.listen,
        args=(self._ct_callback,),
        kwargs={'context': context, 'url': 'wss://certstream.calidog.io/'},
        daemon=True
    )
    thread.start()

```

```
self.logger.info("CT log monitor started – watching for Indian brand phishing domains")
```

```
ct_monitor = CTLogMonitor()
```

3. In main.py startup, extend start_background_tasks():

```
loop = asyncio.get_event_loop()
ct_monitor.start(loop)
logger.info("Proactive intelligence: CT log monitor active")
```

4. In threat_detection dashboard (frontend):

When a threat event has source="ct_log" in qmind_category_scores metadata, display the badge:



PROACTIVELY DETECTED – identified before network impact

This badge MUST appear on all threats discovered via ct_log, paste_monitor, apk_monitor, or domain_monitor sources. It is a core sales differentiator.

5. Add to config.py:

```
CT_MONITOR_ENABLED: bool = True
CT_MONITOR_BRAND_PATTERNS: list = [] # customer-specific additions per tenant
```

Show complete diff.

Prompt 6.2 [NEW v3] — Paste Site Monitor

Read the full contents of:

- kebos-backend/app/crawlers/ct_monitor.py (just created)
- kebos-backend/app/core/config.py

Build paste site monitor – detects Indian credential leaks before they reach HaveIBeenPwned.

1. Add to requirements: beautifulsoup4>=4.12.0 aiohttp>=3.9.0

2. Create kebos-backend/app/crawlers/paste_monitor.py:

```
import asyncio, re, logging, hashlib
from bs4 import BeautifulSoup
from app.core.egress_control import EgressControlledClient
```



```

# Patterns indicating Indian credentials/PII in paste content
INDIAN_CREDENTIAL_PATTERNS = [
    re.compile(r'\b[0-9]{4}\s?[0-9]{4}\s?[0-9]{4}\b'), # Aadhaar-like
    re.compile(r'\b[A-Z]{5}[0-9]{4}[A-Z]\b'), # PAN-like
    re.compile(r'\b[0-9]{11}@([oksbi|okaxis|okicici|okhdfcbank])\b'), # UPI
    re.compile(r'\bIFSC\s*:\s*[A-Z]{4}0[A-Z0-9]{6}\b'), # IFSC code
    re.compile(r'\b[0-9]{8,18}\b\s*\|\s*[0-9]{3,4}\b'), # card-like
]

class PasteMonitor:
    PASTEBIN_ARCHIVE = "https://pastebin.com/archive"

    async def scan_recent_pastes(self):
        """Scan Pastebin public archive for Indian credential patterns."""
        logger = logging.getLogger(__name__)
        async with EgressControlledClient() as client:
            try:
                resp = await client.get(self.PASTEBIN_ARCHIVE, timeout=15.0)
                soup = BeautifulSoup(resp.text, 'html.parser')
                paste_links = [
                    a['href'] for a in soup.find_all('a', href=True)
                    if a['href'].startswith('/') and len(a['href']) == 9
                ][:20] # scan only recent 20 pastes per cycle
                for link in paste_links:
                    await asyncio.sleep(2) # rate limit - be polite
                    raw_url = f"https://pastebin.com/raw{link}"
                    try:
                        paste_resp = await client.get(raw_url, timeout=10.0)
                        await self._analyse_content(paste_resp.text, raw_url,
logger)

                    except Exception as e:
                        logger.debug(f"Failed to fetch paste {link}: {e}")
            except Exception as e:
                logger.error(f"Paste monitor scan failed: {e}")

    async def _analyse_content(self, content: str, source_url: str, logger):
        for pattern in INDIAN_CREDENTIAL_PATTERNS:
            if pattern.search(content):
                content_hash =
hashlib.sha256(content[:500].encode()).hexdigest()
                payload = {
                    "indicator_value": source_url,
                    "indicator_type": "url",
                    "strength": 0.8,

```

```

        "confidence": 0.75,
        "source": "paste_monitor",
        "metadata": {
            "content_hash": content_hash,
            "proactive": True,
            "detected_pattern": pattern.pattern[:50]
        }
    }
    async with EgressControlledClient() as client:
        await client.post(
            "http://qmind:8001/signals/inject",
            json=payload, timeout=5.0
        )
    logger.warning(f"Indian credential pattern found in paste:
{source_url}")
    break # one injection per paste

paste_monitor = PasteMonitor()

3. In main.py startup, add periodic task:
    async def run_paste_monitor():
        while True:
            await paste_monitor.scan_recent_pastes()
            await asyncio.sleep(1800) # scan every 30 minutes

    task = asyncio.create_task(run_paste_monitor(), name="paste-monitor")
    task.add_done_callback(handle_consumer_crash)

Show complete diff.

```

Prompt 6.3 [NEW v3] — Newly Registered Domain Monitor

Read the full contents of:

- kebos-backend/app/crawlers/ (ALL files)
- kebos-backend/app/core/config.py

Build newly registered domain monitoring.

Complements CT log monitoring with WHOIS-based detection.

1. Create kebos-backend/app/crawlers/domain_monitor.py:

```

import asyncio, logging
from datetime import datetime, timedelta
from app.core.egress_control import EgressControlledClient

class NewDomainMonitor:
    """
    Monitors WhoisXML API for newly registered domains matching Indian brands.
    Falls back to free DNS enumeration if API key not configured.
    """
    WHOISXML_URL = "https://newly-registered-domains.whoisxmlapi.com/api/v1"

    async def scan_new_domains(self):
        logger = logging.getLogger(__name__)
        from app.core.config import settings

        if not settings.WHOISXML_API_KEY:
            logger.warning("WHOISXML_API_KEY not set - domain monitor using DNS
fallback")

            await self._dns_fallback_scan(logger)
            return

        async with EgressControlledClient() as client:
            params = {
                "apiKey": settings.WHOISXML_API_KEY,
                "startDate": (datetime.utcnow() -
timedelta(hours=24)).strftime("%Y-%m-%d"),
                "keywords": ",".join(settings.CT_MONITOR_BRAND_PATTERNS[:10]),
            }
            try:
                resp = await client.get(self.WHOISXML_URL, params=params,
timeout=30.0)

                data = resp.json()
                for domain_entry in data.get("domainsList", []):
                    domain = domain_entry.get("domainName", "")
                    if domain:
                        await self._inject_suspicious_domain(domain,
"domain_monitor")

                        logger.info(f"New suspicious domain registered:
{domain}")
            except Exception as e:
                logger.error(f"Domain monitor WhoisXML scan failed: {e}")

    async def _dns_fallback_scan(self, logger):
        """Generate candidate typosquatting domains and check DNS

```

```

resolution."""
    from app.crawlers.ct_monitor import INDIAN_BRAND_PATTERNS
    import socket

    for brand in INDIAN_BRAND_PATTERNS[:5]: # limit to avoid rate issues
        candidates = [
            f"{brand}-secure.com", f"{brand}-login.net",
            f"{brand}-verify.in", f"secure-{brand}.com",
            f"{brand}banking.com", f"{brand}online.in",
        ]
        for candidate in candidates:
            try:
                socket.gethostbyname(candidate)
                # If resolves → domain exists → flag for QMind analysis
                await self._inject_suspicious_domain(candidate,
"domain_monitor")
                logger.warning(f"Typosquatting domain resolves:
{candidate}")
            except socket.gaierror:
                pass # domain doesn't exist – good
                await asyncio.sleep(0.1)

    async def _inject_suspicious_domain(self, domain: str, source: str):
        payload = {
            "indicator_value": domain,
            "indicator_type": "domain",
            "strength": 0.75,
            "confidence": 0.70,
            "source": source,
            "metadata": {"proactive": True, "registration_age": "< 24h"}
        }
        async with EgressControlledClient() as client:
            await client.post("http://qmind:8001/signals/inject",
                             json=payload, timeout=5.0)

```

2. Add to config.py:

```

WHOISXML_API_KEY: str = "" # optional, falls back to DNS scan
DOMAIN_MONITOR_INTERVAL_HOURS: int = 6

```

3. In main.py startup:

```

async def run_domain_monitor():
    while True:
        await domain_monitor.scan_new_domains()
        await asyncio.sleep(settings.DOMAIN_MONITOR_INTERVAL_HOURS * 3600)

```

```
task = asyncio.create_task(run_domain_monitor(), name="domain-monitor")
task.add_done_callback(handle_consumer_crash)
```

Show complete diff.

Prompt 6.4 [NEW v3] — Supplier Trust Engine + Feed Integrity Monitoring

Read the full contents of:

- qmind_enterprise/external_dataset_loader.py
- qmind_enterprise/api/enterprise_api.py
- kebos-backend/app/core/config.py

Add statistical integrity monitoring to all 8 external threat feeds.

A compromised feed supplier can whitelist attacker infrastructure or blacklist legitimate services – degrading detection silently with no visible error.

1. Create qmind_enterprise/supplier_trust/trust_engine.py:

```
from dataclasses import dataclass, field
from datetime import datetime
from scipy import stats
import numpy as np, logging

@dataclass
class FeedBaseline:
    feed_name: str
    historical_scores: list = field(default_factory=list)
    historical_volume: list = field(default_factory=list)
    known_safe_iocs: set = field(default_factory=set)
    known_malicious_iocs: set = field(default_factory=set)
    current_trust_score: float = 1.0
    last_anomaly_at: datetime = None
    quarantined: bool = False

class SupplierTrustEngine:
    def __init__(self):
        self.baselines: dict[str, FeedBaseline] = {}
        self.logger = logging.getLogger(__name__)

    def register_feed(self, feed_name: str):
        self.baselines[feed_name] = FeedBaseline(feed_name=feed_name)
```

```

def check_feed_anomaly(
    self,
    feed_name: str,
    new_data: list,
    new_scores: list
) -> dict:
    baseline = self.baselines.get(feed_name)
    if not baseline or len(baseline.historical_scores) < 10:
        # Not enough history – accept but accumulate
        baseline.historical_scores.extend(new_scores)
        baseline.historical_volume.append(len(new_data))
        return {"anomaly": False, "reason": "insufficient_history"}

    anomalies = {}

    # 1. Volume spike: >3x historical mean
    vol_mean = np.mean(baseline.historical_volume)
    if len(new_data) > vol_mean * 3:
        anomalies["volume_spike"] = {
            "expected": vol_mean, "actual": len(new_data)}

    # 2. Score distribution shift (KS test)
    if len(new_scores) >= 5 and len(baseline.historical_scores) >= 10:
        ks_stat, p_value = stats.ks_2samp(new_scores,
baseline.historical_scores[-100:])
        if p_value < 0.01:
            anomalies["score_distribution_shift"] = {
                "ks_statistic": ks_stat, "p_value": p_value}

    # 3. Known-good IOCs suddenly flagged as malicious
    new_malicious_iocs = set(
        item.get("indicator") for item in new_data
        if item.get("confidence", 0) > 0.7
    )
    false_positives = new_malicious_iocs & baseline.known_safe_iocs
    if false_positives:
        anomalies["known_good_flagged"] = list(false_positives)[:5]

    # 4. Known-bad IOCs suddenly cleared
    new_benign_iocs = set(
        item.get("indicator") for item in new_data
        if item.get("confidence", 0) < 0.2
    )

```

```

cleared_threats = new_benign_iocs & baseline.known_malicious_iocs
if cleared_threats:
    anomalies["known_bad_cleared"] = list(cleared_threats)[:5]

if anomalies:
    self._handle_anomaly(feed_name, anomalies, baseline)
    return {"anomaly": True, "details": anomalies}

# Update trust score (decays slowly toward 1.0 after clean scans)
baseline.current_trust_score = min(1.0, baseline.current_trust_score +
0.01)

baseline.historical_scores.extend(new_scores)
baseline.historical_volume.append(len(new_data))
return {"anomaly": False}

def _handle_anomaly(self, feed_name: str, anomalies: dict, baseline:
FeedBaseline):
    self.logger.warning(
        f"FEED ANOMALY DETECTED: {feed_name} - {list(anomalies.keys())}"
    )
    baseline.current_trust_score *= 0.5
    baseline.last_anomaly_at = datetime.utcnow()
    if baseline.current_trust_score < 0.3:
        baseline.quarantined = True
        self.logger.error(
            f"FEED QUARANTINED: {feed_name} trust score =
{baseline.current_trust_score:.2f}. "
            f"Using cached data only."
        )

def get_qmind_weight(self, feed_name: str) -> float:
    """Returns weight to apply to this feed's signals in QMind analysis."""
    baseline = self.baselines.get(feed_name)
    if not baseline or baseline.quarantined:
        return 0.0
    return baseline.current_trust_score

supplier_trust = SupplierTrustEngine()
# Register all 8 feeds at module load
for feed in ["abuseipdb", "feodo", "malwarebazaar", "nvd", "openphish",
            "phishtank", "urlhaus", "tranco"]:
    supplier_trust.register_feed(feed)

```

2. In `external_dataset_loader.py`, after each feed loads:

- Call `supplier_trust.check_feed_anomaly(feed_name, new_data, scores)`
- If quarantined: use last cached (Dilithium-signed) snapshot
- Pass `supplier_trust.get_qmind_weight(feed_name)` to QMind signal weighting

3. Add to requirements: `scipy>=1.11.0`

Show complete diff.

PHASE 7 — NETWORK TRAFFIC ANALYSIS + ENDPOINT [ALL NEW v3]

Prompt 7.1 [NEW v3] — Zeek NTA Integration

Read the full contents of:

- `kebos-backend/app/threat_detection/` (ALL files)
- `kebos-backend/app/core/config.py`
- `docker-compose.yml`

Integrate Zeek network traffic analysis. This changes Kebos from a tool that requires an existing SIEM to a platform that can directly observe network traffic.

Without this, customers ask "where does the data come from" with no good answer.

1. Add Zeek as a docker-compose service:

```
zeek:
  image: zeekurity/zeek:latest
  network_mode: host # needs access to host network interface
  volumes:
    - ./zeek/logs:/opt/zeek/logs
    - ./zeek/scripts:/opt/zeek/share/zeek/site
  environment:
    ZEEK_INTERFACE: ${NETWORK_INTERFACE:-eth0}
  command: zeek -i ${NETWORK_INTERFACE:-eth0} local
  cap_add:
    - NET_RAW
    - NET_ADMIN
```

2. Create `kebos-backend/app/nta/zeek_log_ingestor.py`:

```
import asyncio, json, logging
from pathlib import Path
from datetime import datetime
```



```

from app.threat_detection.kafka_publisher import threat_publisher

class ZeekLogIngestor:
    """Tails Zeek JSON logs and publishes indicators to threat.indicators."""

    LOG_DIR = Path("/opt/zeek/logs/current")

    LOG_PARSERS = {
        "conn.log": self._parse_conn,
        "dns.log": self._parse_dns,
        "http.log": self._parse_http,
        "ssl.log": self._parse_ssl,
        "notice.log": self._parse_notice,
    }

    def _parse_conn(self, record: dict) -> list:
        """Extract indicators from conn.log entries."""
        indicators = []
        if record.get("id.orig_h") and record.get("orig_bytes", 0) > 10000000:
            indicators.append({
                "value": record["id.orig_h"],
                "type": "ip",
                "context": {"bytes": record["orig_bytes"], "proto":
record.get("proto")}}
            })
        return indicators

    def _parse_dns(self, record: dict) -> list:
        domain = record.get("query", "")
        if domain and record.get("rcode_name") != "NXDOMAIN":
            return [{"value": domain, "type": "domain",
                "context": {"qtype": record.get("qtype_name")}}]
        return []

    def _parse_http(self, record: dict) -> list:
        host = record.get("host", "")
        if host:
            return [{"value": host, "type": "domain",
                "context": {"method": record.get("method"),
                    "uri": record.get("uri", "")[:100]}}]
        return []

    def _parse_ssl(self, record: dict) -> list:
        host = record.get("server_name", "")

```



```

        )
        except (json.JSONDecodeError, KeyError):
            pass
        position = f.tell()
    except Exception as e:
        logger.error(f"Zeek log tail error ({log_name}): {e}")

    async def start(self):
        """Start tailing all Zeek logs concurrently."""
        tasks = [
            asyncio.create_task(self.tail_log(log), name=f"zeek-{log}")
            for log in self.LOG_PARSERS.keys()
        ]
        for task in tasks:
            task.add_done_callback(handle_consumer_crash)

zeek_ingestor = ZeekLogIngestor()

```

3. In main.py startup: await zeek_ingestor.start()

4. Add to config.py:

```

ZEEK_ENABLED: bool = False # True in production deployment
ZEEK_LOG_DIR: str = "/opt/zeek/logs/current"
NETWORK_INTERFACE: str = "eth0"

```

Show complete diff.

Prompt 7.2 [NEW v3] — Endpoint Telemetry Ingestion (Sysmon/auditd)

Read the full contents of:

- kebos-backend/app/nta/ (ALL files just created)
- kebos-backend/app/core/config.py

Add lightweight endpoint telemetry collection.

Ingests Windows Event Logs (Sysmon) and Linux auditd output.

Answers the "do you have EDR" question with a defensible technical answer.

1. Create kebos-backend/app/nta/endpoint_ingestor.py:

```

from fastapi import APIRouter, Depends
from pydantic import BaseModel

```

```

import logging

router = APIRouter(prefix="/api/v1/endpoint", tags=["endpoint"])

class SysmonEvent(BaseModel):
    event_id: int          # 1=ProcessCreate, 3=NetworkConnect, 11=FileCreate
    etc
    computer: str
    timestamp: str
    process_name: str = ""
    command_line: str = ""
    parent_process: str = ""
    destination_ip: str = ""
    destination_port: int = 0
    file_path: str = ""
    tenant_id: str

class AuditdEvent(BaseModel):
    timestamp: str
    hostname: str
    event_type: str        # execve, connect, open, etc
    uid: int
    command: str = ""
    filepath: str = ""
    remote_ip: str = ""
    tenant_id: str

@router.post("/sysmon")
@limiter.limit("1000/minute") # high limit for endpoint telemetry volume
async def ingest_sysmon_event(event: SysmonEvent):
    """Receives Sysmon events from Windows endpoints via lightweight agent."""
    logger = logging.getLogger(__name__)

    # Extract indicators from high-value Sysmon event IDs
    indicators = []
    if event.event_id == 1 and event.command_line: # ProcessCreate
        if any(sus in event.command_line.lower() for sus in
            ['powershell -enc', 'cmd /c', 'wscript', 'cscript', 'mshta',
'regsvr32']):
            indicators.append(("process_cmdline", event.computer, "ip"))
    if event.event_id == 3 and event.destination_ip: # NetworkConnect
        indicators.append(("destination_ip", event.destination_ip, "ip"))

    for _, value, ind_type in indicators:

```

```

        await threat_publisher.publish_indicator(
            indicator_value=value,
            indicator_type=ind_type,
            catboost_score=0.0,
            source_event_id=f"sysmon_{event.computer}_{event.timestamp}",
            metadata={"event_id": event.event_id, "computer": event.computer,
                    "source": "endpoint_sysmon"}
        )
    return {"status": "ingested"}

@router.post("/auditd")
@limiter.limit("1000/minute")
async def ingest_auditd_event(event: AuditdEvent):
    """Receives Linux auditd events from server agents."""
    indicators = []
    if event.remote_ip:
        indicators.append(event.remote_ip)
    for ip in indicators:
        await threat_publisher.publish_indicator(
            indicator_value=ip,
            indicator_type="ip",
            catboost_score=0.0,
            source_event_id=f"auditd_{event.hostname}_{event.timestamp}",
            metadata={"event_type": event.event_type, "uid": event.uid,
                    "source": "endpoint_auditd"}
        )
    return {"status": "ingested"}

```

2. Register endpoint router in main.py.

3. Create a minimal PowerShell agent script (not part of Kebos codebase, saved in docs/agents/sysmon_agent.ps1) that:

- Subscribes to Sysmon ETW events
- POSTs to <https://kebos.customer.com/api/v1/endpoint/sysmon>
- Uses HTTPS with client certificate (mTLS from Prompt 5.2)

Show complete diff.

Prompt 7.3 [NEW v3] — Vulnerability Management Integration

Read the full contents of:

- kebos-backend/app/integrations/ (ALL files)
- kebos-backend/app/core/config.py
- qmind_enterprise/api/enterprise_api.py

Integrate vulnerability scan results. Powers the CVE_Exploitation QMind category with real asset context. Enables the correlation:

"NVD says CVE-2024-XXXX is being exploited + your environment has 47 vulnerable servers"

1. Create kebos-backend/app/integrations/vuln_management.py:

```
from pydantic import BaseModel
from typing import Optional
import httpx

class VulnerabilityFinding(BaseModel):
    asset_ip: str
    asset_hostname: str = ""
    cve_id: str
    severity: str          # CRITICAL, HIGH, MEDIUM, LOW
    cvss_score: float
    plugin_name: str = ""
    tenant_id: str

class NessusImporter:
    """Imports vulnerability findings from Nessus/Tenable."""
    def __init__(self, access_key: str, secret_key: str, base_url: str):
        self.access_key = access_key
        self.secret_key = secret_key
        self.base_url = base_url

    async def get_latest_scan_findings(self, scan_id: int) ->
list[VulnerabilityFinding]:
        headers = {
            "X-ApiKeys": f"accessKey={self.access_key}; secretKey=
{self.secret_key}"
        }
        async with httpx.AsyncClient(base_url=self.base_url) as client:
            resp = await client.get(f"/scans/{scan_id}/hosts", headers=headers)
            findings = []
            for host in resp.json().get("hosts", []):
                host_id = host["host_id"]
                vuln_resp = await client.get(
                    f"/scans/{scan_id}/hosts/{host_id}", headers=headers)
```

```

        for vuln in vuln_resp.json().get("vulnerabilities", []):
            if vuln.get("severity", 0) >= 3: # HIGH or CRITICAL only
                findings.append(VulnerabilityFinding(
                    asset_ip=host.get("hostname", ""),
                    cve_id=vuln.get("plugin_name", ""),
                    severity=["INFO", "LOW", "MEDIUM", "HIGH", "CRITICAL"]
                        [vuln["severity"]],

                    cvss_score=vuln.get("score", 0.0),
                    plugin_name=vuln.get("plugin_name", ""),
                    tenant_id="" # set by caller

                ))
        return findings

```

2. Create POST /api/v1/vuln/import endpoint:

Body: list[VulnerabilityFinding]

Action:

- a. Store in vuln_findings table (create model + Alembic migration)
- b. Cross-reference with NVD feed in QMind:

For each CVE_ID in findings, check if QMind has active exploitation signals
- c. If CVE has active exploitation + asset in customer environment:

Inject HIGH confidence signal to QMind:

```
source="vulnerability_scan", confidence=0.80,
metadata={"cve": cve_id, "affected_assets": count, "severity": severity}
```

3. Create POST /api/v1/vuln/nessus-sync endpoint:

Triggers Nessus API pull for configured scan IDs.

Requires: NESSUS_ACCESS_KEY, NESSUS_SECRET_KEY, NESSUS_BASE_URL in config.

4. Alembic migration: create_vuln_findings_table

Columns: id, asset_ip, asset_hostname, cve_id, severity, cvss_score,
tenant_id, scan_date, qmind_correlated (boolean), created_at

Show complete diff.

PHASE 8 — THREAT INTELLIGENCE PLATFORM [ALL NEW v3]

Prompt 8.1 [NEW v3] — TIP Core: IOC Database + MITRE ATT&CK Mapping

Read the full contents of:

- kebos-backend/app/threat_detection/models.py
- kebos-backend/app/integrations/stix_taxii.py

- kebos-backend/alembic/versions/ (most recent migration)

Build the Threat Intelligence Platform layer – where analysts store what they learn,

link IOCs into campaigns, and tag MITRE ATT&CK techniques.

1. Alembic migration: create_tip_tables

```
CREATE TABLE ioc_records (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    indicator_value TEXT NOT NULL,  
    indicator_type VARCHAR(20) NOT NULL, -- ip, hash, url, domain, email  
    source TEXT NOT NULL, -- feed name, analyst, honeypot,  
crawler  
    confidence FLOAT NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'active', -- active, expired,  
false_positive  
    first_seen TIMESTAMP NOT NULL DEFAULT NOW(),  
    last_seen TIMESTAMP NOT NULL DEFAULT NOW(),  
    qmind_lead_category VARCHAR(64),  
    qmind_category_scores JSONB,  
    mitre_techniques TEXT[], -- e.g. ['T1566', 'T1059.001']  
    campaign_id UUID,  
    analyst_notes TEXT,  
    tags TEXT[],  
    created_by UUID REFERENCES users(id),  
    FOREIGN KEY (tenant_id) REFERENCES tenants(id)  
);
```

```
CREATE TABLE campaigns (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    name VARCHAR(255) NOT NULL,  
    description TEXT,  
    threat_actor VARCHAR(255), -- e.g. 'SideWinder', 'Lazarus'  
    first_seen TIMESTAMP,  
    last_seen TIMESTAMP,  
    status VARCHAR(20) DEFAULT 'active',  
    mitre_techniques TEXT[],  
    created_by UUID REFERENCES users(id),  
    FOREIGN KEY (tenant_id) REFERENCES tenants(id)  
);
```



```
CREATE TABLE mitre_techniques (  
  technique_id VARCHAR(20) PRIMARY KEY,    -- e.g. T1566.001  
  name VARCHAR(255) NOT NULL,  
  tactic VARCHAR(100) NOT NULL,            -- e.g. Initial Access  
  description TEXT,  
  url TEXT  
);
```

2. Create kebos-backend/app/tip/router.py:

```
GET    /api/v1/tip/iocs          - list IOCs with filter/sort/page  
POST   /api/v1/tip/iocs          - create/update IOC record  
GET    /api/v1/tip/iocs/{id}     - get IOC detail  
PUT    /api/v1/tip/iocs/{id}/notes - analyst annotations  
PUT    /api/v1/tip/iocs/{id}/mitre - tag ATT&CK techniques  
PUT    /api/v1/tip/iocs/{id}/campaign - link IOC to campaign  
GET    /api/v1/tip/campaigns     - list campaigns  
POST   /api/v1/tip/campaigns     - create campaign  
GET    /api/v1/tip/mitre/techniques - search ATT&CK technique library
```

3. Auto-populate TIP from QMind results:

```
In update_threat_with_qmind_result() (threat_detection/services.py),  
after updating threat record, create/update an IOC record in tip.ioc_records:  
- If IOC record exists for this indicator: update last_seen, confidence  
- If not: create new record with QMind output  
This means every QMind-analysed indicator is automatically a TIP entry.
```

4. Seed mitre_techniques table with the 193 ATT&CK Enterprise techniques relevant to Indian threat landscape. Create a migration data file: docs/mitre_seed.json with at minimum the top 30 techniques seen in CERT-In advisories (T1566, T1059, T1078, T1190, T1133, T1486, etc.)

Show complete diff.

Prompt 8.2 [NEW v3] — Threat Actor Profiling

Read the full contents of:

- kebos-backend/app/tip/router.py (just created)
- kebos-backend/app/tip/ (ALL files)

Add Indian-specific threat actor profiles. When QMind classifies a threat,

link it to known actor TTPs to transform a generic alert into specific intelligence.

1. Alembic migration: create_threat_actor_profiles

```
CREATE TABLE threat_actors (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    name VARCHAR(255) NOT NULL UNIQUE,      -- e.g. 'SideWinder', 'Lazarus'  
    aliases TEXT[],                          -- e.g. ['APT-C-17', 'Rattlesnake']  
    suspected_origin VARCHAR(100),  
    primary_targets TEXT[],                  -- e.g. ['Indian BFSI', 'Indian  
Defence']  
    active_since DATE,  
    last_observed DATE,  
    mitre_techniques TEXT[],  
    known_c2_patterns JSONB,                -- IP ranges, domain patterns  
    known_malware_families TEXT[],  
    cert_in_references TEXT[],              -- CERT-In advisory IDs  
    description TEXT  
);
```

2. Seed with Indian-relevant threat actors:

INSERT threat actor profiles for:

- SideWinder (targets Indian defence, government, BFSI)
Techniques: T1566.001, T1204.002, T1059.005, T1053.005
Known targets: HAL, BEL, DRDO, Indian Navy, Indian Finance Ministry
- Lazarus (targets Indian crypto, BFSI)
Techniques: T1190, T1133, T1486, T1041
- Bitter (APT-C-08, targets India/Pakistan border region)
- SilverTerrier (Nigerian BEC targeting Indian companies)
- REvil affiliates observed in Indian infrastructure

3. Create kebos-backend/app/tip/actor_matcher.py:

```
class ThreatActorMatcher:  
    async def match_to_actor(  
        self,  
        indicator: str,  
        qmind_result: dict,  
        ioc_record: dict  
    ) -> list[dict]:  
        """  
        Returns list of matching threat actor profiles for an IOC.  
        Matching logic:
```

```

1. If lead_category == "C2_Infrastructure": check against
known_c2_patterns

2. If mitre_techniques tagged on IOC: intersect with actor technique
lists

3. If campaign_id set: use campaign's threat_actor field directly
"""

matches = []
# Implement matching against threat_actors table
return matches

4. In update_threat_with_qmind_result(), after TIP IOC creation:
    actor_matches = await actor_matcher.match_to_actor(indicator, qmind_result,
ioc_record)
    If matches found: add actor name to IOC record tags and SOC report context

5. Add to SOC report generator prompt:
    If threat actor matched: include in THREAT_CONTEXT section:
    "This IOC matches known TTPs of {actor_name}, a {origin} threat actor
    historically targeting {targets}. Relevant CERT-In advisories: {refs}"

Show complete diff.

```

Prompt 8.3 [NEW v3] — STIX Export + Analyst Feedback Loop

```

Read the full contents of:
- kebos-backend/app/integrations/stix_taxii.py
- kebos-backend/app/tip/ (ALL files)

1. Enhance STIX export to include TIP data:
    - Each STIX Indicator: include mitre_techniques as kill_chain_phases
    - Each STIX Indicator: include x_kebos_actor (threat actor name if matched)
    - Each STIX Indicator: include x_kebos_pqc_signature (Dilithium-3 signature)
    - Each STIX Indicator: include x_kebos_campaign (campaign name if linked)

2. Create analyst.feedback Kafka topic consumer:

Create kebos-backend/app/tip/feedback_consumer.py:

async def consume_analyst_feedback():
    """
    Consumes analyst correction events and queues for CatBoost retraining.

```

This is the feedback loop that enables India-specific fine-tuning.

```
"""
```

```
consumer = AIOKafkaConsumer(
    "analyst.feedback",
    bootstrap_servers=settings.KAFKA_BOOTSTRAP_SERVERS,
    group_id="kebos-feedback-consumer"
)
await consumer.start()
try:
    async for message in consumer:
        feedback = message.value
        # feedback schema:
        # {ioc_id, indicator, qmind_category, analyst_category,
        #  is_correct: bool, analyst_id, tenant_id, timestamp}
        await store_feedback_record(feedback)
        if not feedback["is_correct"]:
            logger.info(
                f"Analyst correction: QMind said
{feedback['qmind_category']}, "
                f"analyst says {feedback['analyst_category']} - "
                f"queued for CatBoost retraining dataset"
            )
finally:
    await consumer.stop()
```

3. Add feedback endpoint for analyst UI:

POST /api/v1/tip/feedback

Body: {ioc_id, is_correct, analyst_category (if not correct)}

Action: publish to analyst.feedback Kafka topic

4. Create feedback_records table:

Alembic migration: create_feedback_records_table

Columns: id, ioc_id, indicator, qmind_category, analyst_category,
is_correct, analyst_id, tenant_id, timestamp

This table IS the CatBoost retraining dataset – exported periodically.

5. In main.py startup:

```
task = asyncio.create_task(consume_analyst_feedback(), name="feedback-
consumer")
task.add_done_callback(handle_consumer_crash)
```

Show complete diff.

PHASE 9 — UEBA + BEHAVIOURAL ANALYTICS [ALL NEW v3]

Prompt 9.1 [NEW v3] — UEBA Baseline Engine

Read the full contents of:

- kebos-backend/app/auth/models.py
- kebos-backend/app/threat_detection/models.py
- kebos-backend/app/core/config.py

Build UEBA — detects insider threats via behavioural baselines.
This is Patent Claim 3: server-side behavioural biometrics via
API request graph temporal analysis.

1. Alembic migration: create_ueba_tables

```
CREATE TABLE user_behaviour_baselines (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID NOT NULL REFERENCES users(id),  
    tenant_id UUID NOT NULL,  
    metric_name VARCHAR(100) NOT NULL, -- 'daily_api_calls',  
    'avg_session_hours', etc  
    baseline_mean FLOAT NOT NULL,  
    baseline_std FLOAT NOT NULL,  
    sample_count INTEGER NOT NULL DEFAULT 0,  
    last_updated TIMESTAMP NOT NULL DEFAULT NOW(),  
    UNIQUE (user_id, metric_name)  
);  
  
CREATE TABLE ueba_anomaly_events (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID NOT NULL,  
    tenant_id UUID NOT NULL,  
    metric_name VARCHAR(100) NOT NULL,  
    observed_value FLOAT NOT NULL,  
    baseline_mean FLOAT NOT NULL,  
    z_score FLOAT NOT NULL,  
    risk_level VARCHAR(20) NOT NULL, -- LOW, MEDIUM, HIGH, CRITICAL  
    detected_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

2. Create kebos-backend/app/ueba/baseline_engine.py:

```
import numpy as np
```

```

from datetime import datetime, timedelta

UEBA_METRICS = [
    "daily_api_calls",
    "daily_data_volume_mb",
    "avg_session_duration_minutes",
    "off_hours_access_ratio",      # ratio of requests outside 6am-10pm IST
    "unique_endpoints_accessed",
    "failed_auth_attempts",
    "report_downloads_per_day",
    "admin_endpoint_access_ratio",
]

class UEBABaselineEngine:
    def __init__(self, db):
        self.db = db

    async def update_baseline(self, user_id: str, tenant_id: str, metric: str,
value: float):
        """Online update of Welford's algorithm for incremental
mean/variance."""
        baseline = await self.db.get_baseline(user_id, metric)
        if baseline is None:
            await self.db.create_baseline(user_id, tenant_id, metric, value, 0,
1)

            return

        n = baseline.sample_count + 1
        old_mean = baseline.baseline_mean
        new_mean = old_mean + (value - old_mean) / n
        old_std = baseline.baseline_std
        # Welford's online variance
        if n > 1:
            new_variance = ((n-2) * old_std**2 + (value - old_mean) * (value -
new_mean)) / (n-1)
            new_std = max(np.sqrt(new_variance), 0.01) # minimum std to avoid
zero-division
        else:
            new_std = old_std
        await self.db.update_baseline(user_id, metric, new_mean, new_std, n)

    async def score_behaviour(self, user_id: str, tenant_id: str, observations:
dict) -> dict:
        """

```

```

Score current behaviour against baseline. Returns risk per metric.
observations = {"metric_name": observed_value, ...}
"""
anomalies = {}
for metric, value in observations.items():
    baseline = await self.db.get_baseline(user_id, metric)
    if not baseline or baseline.sample_count < 7:
        continue # need at least 7 days of data
    z_score = abs(value - baseline.baseline_mean) /
max(baseline.baseline_std, 0.01)
    if z_score > 4.0:
        risk = "CRITICAL"
    elif z_score > 3.0:
        risk = "HIGH"
    elif z_score > 2.0:
        risk = "MEDIUM"
    else:
        continue # within normal range

    anomalies[metric] = {
        "z_score": z_score, "risk": risk,
        "observed": value, "baseline_mean": baseline.baseline_mean
    }
    # Store anomaly event
    await self.db.create_anomaly_event(user_id, tenant_id, metric,
value,
baseline.baseline_mean, z_score,
risk)

    # If HIGH or CRITICAL: inject to QMind as Insider_Threat signal
    if risk in ("HIGH", "CRITICAL"):
        await self._inject_insider_threat(user_id, tenant_id, metric,
z_score)

    return anomalies

    async def _inject_insider_threat(self, user_id, tenant_id, metric,
z_score):
        confidence = min(0.95, 0.5 + (z_score - 3.0) * 0.1)
        payload = {
            "indicator_value": user_id,
            "indicator_type": "ip", # use user identifier as indicator
            "strength": 0.8,
            "confidence": confidence,
            "source": "analyst",
            "metadata": {

```

```
        "ueba_metric": metric,
        "z_score": z_score,
        "tenant_id": tenant_id,
        "source_system": "ueba"
    }
}

async with EgressControlledClient() as client:
    await client.post("http://qmind:8001/signals/inject",
                     json=payload, timeout=5.0)
```

3. Register UEBA middleware in main.py to collect metrics per authenticated request.

4. Register endpoint: GET /api/v1/ueba/anomalies (ADMIN, ANALYST only)

Show complete diff.

Prompt 9.2 [NEW v3] — UEBA Dashboard + Insider Threat Alerts

Read the full contents of:

- kebos-backend/app/ueba/ (ALL files)
- kebos-frontend/src/ (relevant dashboard components)

1. Add UEBA endpoints:

```
GET /api/v1/ueba/users/{user_id}/baseline - current baseline per metric
GET /api/v1/ueba/users/{user_id}/anomalies - recent anomaly history
GET /api/v1/ueba/risk-summary - top N users by risk score (ADMIN only)
```

2. Frontend UEBA page:

- Risk heatmap: users × metrics grid, color-coded by z-score
- User profile: baseline chart per metric with current observation overlaid
- Anomaly feed: real-time WebSocket updates on HIGH/CRITICAL anomalies
- Filter by: department, role, date range, risk level

3. Insider threat alert in analyst queue:

When UEBA injects to QMind with Insider_Threat category:

- Alert shows user details (anonymised if CONFIDENTIAL classification)
- Shows which metric triggered: "API calls: 340 (baseline: 45 ± 12)"
- Links to UEBA user profile

4. Privacy controls:

- UEBA data is CONFIDENTIAL classification – never goes to external LLM

- Anomaly events are masked in multi-tenant view (only own tenant visible)
- Analyst sees user_id and role, NOT full name unless they have ADMIN role

Show complete diff.

PHASE 10 — CASE MANAGEMENT + PLAYBOOKS [ALL NEW v3]

Prompt 10.1 [NEW v3] — Case Management Core

Read the full contents of:

- kebos-backend/app/threat_detection/models.py
- kebos-backend/app/tip/router.py
- kebos-backend/alembic/versions/ (most recent migration)

Build case management — tracks incidents from detection through resolution.
Required for CERT-In compliance (must document all incident response actions).

1. Alembic migration: create_cases_table

```
CREATE TABLE cases (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL,  
    title VARCHAR(255) NOT NULL,  
    description TEXT,  
    status VARCHAR(30) NOT NULL DEFAULT 'open',  
    -- open, in_progress, pending_approval, resolved, closed, false_positive  
    severity VARCHAR(20) NOT NULL,    -- CRITICAL, HIGH, MEDIUM, LOW  
    assigned_to UUID REFERENCES users(id),  
    created_by UUID REFERENCES users(id),  
    threat_event_ids UUID[],           -- linked ThreatEvent records  
    ioc_ids UUID[],                   -- linked IOC records  
    cert_in_report_id UUID,           -- linked CERT-In report if generated  
    timeline_events JSONB DEFAULT '[]', -- ordered list of actions taken  
    resolution_notes TEXT,  
    opened_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    resolved_at TIMESTAMP,  
    cert_in_deadline TIMESTAMP,        -- opened_at + 6 hours  
    sla_breached BOOLEAN DEFAULT FALSE,  
    FOREIGN KEY (tenant_id) REFERENCES tenants(id)  
);
```

2. Create kebos-backend/app/cases/router.py:

```
POST /api/v1/cases                - create case (auto from
CONFIRMED_THREAT)
GET  /api/v1/cases                - list cases with filter/sort
GET  /api/v1/cases/{id}          - case detail
PUT  /api/v1/cases/{id}/status    - update status
PUT  /api/v1/cases/{id}/assign    - assign analyst
POST /api/v1/cases/{id}/timeline  - add timeline event
POST /api/v1/cases/{id}/evidence  - attach evidence (IOC, screenshot,
note)
POST /api/v1/cases/{id}/generate-report - trigger CERT-In report generation
```

3. Auto-create case on CONFIRMED_THREAT:

In `update_threat_with_qmind_result()`, when `confidence >= 0.75`:

- Auto-create case with `status="open"`
- Set `cert_in_deadline = NOW() + 6 hours`
- Push WebSocket event: `{"event": "case_created", "case_id": ..., "deadline": ...}`

4. SLA monitoring background task:

```
async def monitor_cert_in_sla():
    while True:
        await asyncio.sleep(300) # check every 5 minutes
        overdue = await db.cases.get_overdue_open(cutoff=datetime.utcnow())
        for case in overdue:
            await db.cases.update(case.id, sla_breached=True)
            await notify_analyst(case, "CERT-In 6-hour deadline approaching")
            logger.error(f"CERT-In SLA BREACH: Case {case.id} - {case.title}")
        task = asyncio.create_task(monitor_cert_in_sla(), name="cert-in-sla-monitor")
        task.add_done_callback(handle_consumer_crash)
```

Show complete diff.

Prompt 10.2 [NEW v3] — Automated Playbooks

Read the full contents of:

- kebos-backend/app/cases/router.py (just created)
- kebos-backend/app/simulation/digital_twin.py
- kebos-backend/app/deception/honeygrid.py

Add automated playbook execution.

REVERSIBLE actions: auto-execute + notify analyst.

IRREVERSIBLE actions: Digital Twin dry-run → analyst approval required.

1. Create kebos-backend/app/playbooks/engine.py:

```
from enum import Enum

class PlaybookAction(str, Enum):
    BLOCK_IP_FIREWALL = "block_ip_firewall" # IRREVERSIBLE
    ISOLATE_HOST = "isolate_host" # IRREVERSIBLE
    REVOKE_USER_SESSION = "revoke_user_session" # REVERSIBLE
    DISABLE_USER_ACCOUNT = "disable_user_account" # IRREVERSIBLE
    DEPLOY_HONEYPOT = "deploy_honeypot" # REVERSIBLE
    SEND_ANALYST_ALERT = "send_analyst_alert" # REVERSIBLE
    CREATE_CERT_IN_DRAFT = "create_cert_in_draft" # REVERSIBLE
    INCREASE_LOGGING = "increase_logging" # REVERSIBLE

REVERSIBLE_ACTIONS = {
    PlaybookAction.REVOKE_USER_SESSION,
    PlaybookAction.DEPLOY_HONEYPOT,
    PlaybookAction.SEND_ANALYST_ALERT,
    PlaybookAction.CREATE_CERT_IN_DRAFT,
    PlaybookAction.INCREASE_LOGGING,
}

class PlaybookEngine:
    async def execute(
        self,
        action: PlaybookAction,
        target: str,
        case_id: str,
        confidence: float,
        tenant_id: str
    ):
        is_reversible = action in REVERSIBLE_ACTIONS

        if is_reversible:
            result = await self._execute_action(action, target, tenant_id)
            await self._log_to_case(case_id, action, target, result,
"AUTO_EXECUTED")
            await self._notify_analyst(case_id, action, target, result)
            return {"status": "executed", "action": action, "result": result}
        else:
```

```

        # IRREVERSIBLE: Digital Twin first
        sim = await digital_twin.simulate_action(
            action_type=action.value, target=target)
        if sim.safe_to_execute:
            # Still require analyst approval for irreversible actions
            approval_task = await self._create_approval_task(
                case_id, action, target, sim)
            return {"status": "pending_approval", "approval_id":
approval_task.id,
                    "simulation": sim}
        else:
            await self._log_to_case(case_id, action, target, sim,
"BLOCKED_BY_SIMULATION")
            return {"status": "blocked", "reason": sim.recommendation,
                    "simulation": sim}

    async def _execute_action(self, action, target, tenant_id):
        if action == PlaybookAction.REVOKE_USER_SESSION:
            # Flush JTI blacklist entry for user
            await redis.delete(f"session:{tenant_id}:{target}")
            return {"revoked": True}
        elif action == PlaybookAction.DEPLOY_HONEYPOT:
            return await honeygrid.deploy_honeypot(target, "auto_playbook",
0.75)
        elif action == PlaybookAction.CREATE_CERT_IN_DRAFT:
            return {"draft_created": True, "url": f"/reports/cert-
in/draft/{target}"}
        # ... implement remaining actions
        return {}

```

2. Standard playbooks – define in config:

```

STANDARD_PLAYBOOKS = {
    "phishing_confirmed": [
        PlaybookAction.CREATE_CERT_IN_DRAFT,
        PlaybookAction.SEND_ANALYST_ALERT,
        PlaybookAction.DEPLOY_HONEYPOT,
    ],
    "c2_confirmed": [
        PlaybookAction.CREATE_CERT_IN_DRAFT,
        PlaybookAction.SEND_ANALYST_ALERT,
        PlaybookAction.INCREASE_LOGGING,
        PlaybookAction.ISOLATE_HOST, # IRREVERSIBLE – requires approval
    ],
    "insider_threat": [

```

```

        PlaybookAction.SEND_ANALYST_ALERT,
        PlaybookAction.INCREASE_LOGGING,
        PlaybookAction.REVOKE_USER_SESSION,
    ],
}

```

```

3. Trigger playbook in update_threat_with_qmind_result() on CONFIRMED_THREAT:
    playbook = STANDARD_PLAYBOOKS.get(lead_category.lower(), [])
    for action in playbook:
        await playbook_engine.execute(action, indicator, case_id, confidence,
tenant_id)

```

Show complete diff.

Prompt 10.3 [NEW v3] — Analyst Approval Workflow

Read the full contents of:

- kebos-backend/app/playbooks/engine.py (just created)
- kebos-backend/app/cases/router.py

Add approval workflow for IRREVERSIBLE playbook actions.

1. Alembic migration: create_approval_tasks_table

```

CREATE TABLE approval_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL REFERENCES cases(id),
    action_type VARCHAR(100) NOT NULL,
    target VARCHAR(255) NOT NULL,
    simulation_result JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending', -- pending, approved,
rejected
    assigned_to UUID REFERENCES users(id),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    decided_at TIMESTAMP,
    decision_notes TEXT,
    executed_at TIMESTAMP,
    tenant_id UUID NOT NULL
);

```

2. Add endpoints to cases/router.py:

```
GET /api/v1/approvals - list pending approvals for analyst
POST /api/v1/approvals/{id}/approve - approve action
POST /api/v1/approvals/{id}/reject - reject action
Both require ANALYST or ADMIN role.
Both log decision to case timeline.
```

3. On approval:

- Execute the action via `playbook_engine._execute_action()`
- Update `approval_task`: `status=approved`, `decided_at`, `executed_at`
- Add to case timeline: `"ANALYST_APPROVED: {action} on {target}"`
- Push WebSocket event to dashboard

4. On rejection:

- Update `approval_task`: `status=rejected`, `decision_notes`
- Add to case timeline: `"ANALYST_REJECTED: {action} on {target} - {notes}"`
- No action executed

5. Frontend: approval queue card in analyst dashboard

Shows: action type, target, simulation result, `safe_to_execute` flag, estimated FP count, "Approve" and "Reject" buttons with notes field.

Show complete diff.

PHASE 11 — MULTI-TENANCY + FORENSICS [ALL NEW v3]

Prompt 11.1 [NEW v3] — Multi-tenant Admin UI + Tenant Management

Read the full contents of:

- `kebos-backend/app/core/tenant_context.py` (from Prompt 5.3)
- `kebos-backend/app/auth/models.py`
- `kebos-frontend/src/` (admin pages if any)

Complete the multi-tenant implementation with admin management.

1. Create `kebos-backend/app/admin/router.py` (ADMIN role only):

```
POST /api/v1/admin/tenants - create tenant
GET /api/v1/admin/tenants - list all tenants
GET /api/v1/admin/tenants/{id} - tenant detail + stats
PUT /api/v1/admin/tenants/{id} - update tenant settings
POST /api/v1/admin/tenants/{id}/users - invite user to tenant
```

```
GET /api/v1/admin/tenants/{id}/usage - usage statistics
```

2. Tenant auth policy enforcement:

In dependencies.py, after authenticating user:

- Load tenant from DB using user.tenant_id
- Enforce TENANT_AUTH_POLICIES[tenant.tenant_type]:
 - If require_fido2: True and user used TOTP → return HTTP 403
 - Apply session duration limit from policy

3. Tenant-specific QMind thresholds:

When update_threat_with_qmind_result() determines status:

- Load tenant.auth_policy for threat thresholds
- Use THRESHOLDS[tenant.tenant_type]["confirmed_threat"] instead of hardcoded 0.75

4. Frontend tenant switcher (for admin users):

- Dropdown in nav bar (visible only to ADMIN)
- Switches tenant_id context for viewing data
- Does NOT bypass RLS – requests are always scoped to switched tenant

Show complete diff.

Prompt 11.2 [NEW v3] — Forensic Timeline Reconstruction

Read the full contents of:

- kebos-backend/app/cases/router.py
- kebos-backend/app/threat_detection/models.py
- kebos-backend/app/audit_logger/ (ALL files)

Add forensic timeline reconstruction using TimescaleDB.

After confirming a threat, automatically reconstruct everything that happened: who, what, when, from where, and what data was accessed.

1. Create kebos-backend/app/forensics/timeline.py:

```
from datetime import datetime, timedelta
```

```
class ForensicTimeline:
```

```
    def __init__(self, db):
        self.db = db
```

```

async def reconstruct(
    self,
    indicator: str,
    tenant_id: str,
    window_hours: int = 72
) -> dict:
    """
    Reconstructs complete incident timeline.
    Uses TimescaleDB time_bucket() for efficient temporal queries.
    """
    since = datetime.utcnow() - timedelta(hours=window_hours)

    events = []

    # 1. Auth events involving this indicator
    auth_events = await self.db.execute("""
        SELECT timestamp, user_id, action, ip_address, success
        FROM audit_logs
        WHERE tenant_id = :tenant_id
        AND (ip_address = :indicator OR details::text LIKE
:indicator_pattern)
        AND timestamp >= :since
        ORDER BY timestamp ASC
    """, {"tenant_id": tenant_id, "indicator": indicator,
        "indicator_pattern": f"%{indicator}%", "since": since})
    events.extend([**e, "category": "auth"} for e in auth_events])

    # 2. Threat events involving this indicator
    threat_events = await self.db.execute("""
        SELECT timestamp, id as event_id, qmind_lead_category,
            qmind_confidence, qmind_advisory, status
        FROM threat_events
        WHERE tenant_id = :tenant_id
        AND indicator_value = :indicator
        AND created_at >= :since
        ORDER BY created_at ASC
    """, {"tenant_id": tenant_id, "indicator": indicator, "since": since})
    events.extend([**e, "category": "threat"} for e in threat_events])

    # 3. Network events from Zeek (if NTA enabled)
    if settings.ZEEK_ENABLED:
        network_events = await self.db.execute("""
            SELECT time as timestamp, source_ip, dest_ip, protocol,
                bytes, packets, action

```



```

        FROM network_flows
        WHERE tenant_id = :tenant_id
        AND (source_ip = :indicator OR dest_ip = :indicator)
        AND time >= :since
        ORDER BY time ASC
        """ , {"tenant_id": tenant_id, "indicator": indicator, "since":
since})

        events.extend([{"*e, "category": "network"} for e in
network_events])

# 4. Honeypot interactions
honeypot_events = await self.db.execute("""
        SELECT created_at as timestamp, attacker_ip, interaction_type,
               commands_executed, confidence
        FROM honeypot_interactions
        WHERE tenant_id = :tenant_id
        AND attacker_ip = :indicator
        AND created_at >= :since
        ORDER BY created_at ASC
        """, {"tenant_id": tenant_id, "indicator": indicator, "since": since})
events.extend([{"*e, "category": "honeypot"} for e in honeypot_events])

# Sort all events chronologically
events.sort(key=lambda x: x.get("timestamp", datetime.min))

return {
    "indicator": indicator,
    "window_hours": window_hours,
    "total_events": len(events),
    "first_seen": events[0]["timestamp"] if events else None,
    "last_seen":  events[-1]["timestamp"] if events else None,
    "timeline": events,
    "summary": self._summarise(events)
}

def _summarise(self, events: list) -> str:
    categories = {}
    for e in events:
        cat = e.get("category", "unknown")
        categories[cat] = categories.get(cat, 0) + 1
    parts = [f"{v} {k} events" for k, v in categories.items()]
    return f"Timeline: {' , '.join(parts)}"

```

2. Add endpoint to cases/router.py:

```
GET /api/v1/cases/{id}/timeline/forensic
```

```
Calls ForensicTimeline.reconstruct() using the case's linked threat events
```

3. Frontend: forensic timeline view in case detail page

- Chronological event stream with category colour coding
- Auth events (blue), Threat events (red), Network events (orange), Honeypot (purple)
- Expandable event detail
- Export as PDF (included in CERT-In report generation)

Show complete diff.

PHASE 12 — SUPPLIER TRUST + GRACEFUL DEGRADATION [ALL NEW v3]

Prompt 12.1 [NEW v3] — Vault Breach Response + Secret Rotation

Read the full contents of:

- kebos-backend/app/core/config.py
- kebos-backend/app/auth/services.py
- kebos-backend/app/main.py

Implement emergency secret rotation for Vault breach scenario.

This is the most catastrophic single-point breach in the architecture.

1. Create kebos-backend/app/security/breach_response.py:

```
class VaultBreachResponse:
    """
    Emergency rotation of all secrets on suspected Vault compromise.
    Target completion: < 5 minutes from trigger.
    """
    def __init__(self, vault_client, redis_client, db):
        self.vault = vault_client
        self.redis = redis_client
        self.db = db
        self.logger = logging.getLogger(__name__)

    async def emergency_rotation(self, triggered_by: str, reason: str):
        start_time = datetime.utcnow()
        self.logger.critical(
            f"EMERGENCY ROTATION TRIGGERED by {triggered_by}: {reason}")
```

```

steps = [
    ("Generate new RSA keypair", self._rotate_rsa_keypair),
    ("Flush all Redis JTI tokens", self._flush_all_sessions),
    ("Rotate DB password", self._rotate_db_password),
    ("Rotate Kafka credentials", self._rotate_kafka_creds),
    ("Rotate external API keys", self._rotate_api_keys),
    ("Alert administrators OOB", self._alert_admins),
    ("Log rotation audit record", self._log_rotation),
]

results = {}
for step_name, step_fn in steps:
    try:
        await step_fn()
        results[step_name] = "OK"
        self.logger.info(f"Emergency rotation step complete:
{step_name}")
    except Exception as e:
        results[step_name] = f"FAILED: {e}"
        self.logger.error(f"Emergency rotation step failed:
{step_name}: {e}")

    elapsed = (datetime.utcnow() - start_time).total_seconds()
    self.logger.critical(
        f"Emergency rotation complete in {elapsed:.1f}s. Results:
{results}")
    if elapsed > 300:
        self.logger.error(f"EMERGENCY ROTATION EXCEEDED 5-MINUTE TARGET:
{elapsed:.1f}s")
    return results

async def _rotate_rsa_keypair(self):
    from app.core.rsa_keys import load_or_generate_keypair
    # Generate new keypair
    # Store in Vault at new path
    # Update settings to use new keys
    # Old keys remain valid until all tokens expire (15 min max)
    pass # implement against your Vault client

async def _flush_all_sessions(self):
    """Delete ALL JTI blacklist keys – forces re-auth for everyone."""
    keys = await self.redis.keys("jti:*")
    if keys:

```

```

        await self.redis.delete(*keys)
        self.logger.info(f"Flushed {len(keys)} JTI tokens – all users logged
out")

    async def _alert_admins(self):
        """Alert via out-of-band channel (NOT email – email may be
compromised)."""
        # Implement: SMS via Twilio, PagerDuty webhook, or phone call
        # Never use the compromised system's own notification channel
        pass

2. Add trigger endpoint (ADMIN only, requires FIDO2):
POST /api/v1/security/emergency-rotation
Body: {"reason": str}
Requires: user.role == ADMIN and fido2_authenticated == True

3. Add Vault anomaly monitoring in main.py:
Monitor Vault audit log for:
- Bulk secret reads (>20 secrets read in < 60 seconds)
- Reads from unexpected IP addresses
- Auth failures followed by success from same IP
If anomaly detected: auto-trigger emergency_rotation() with
reason="vault_anomaly_detected"

Show complete diff.

```

Prompt 12.2 [NEW v3] — Graceful Degradation for All Dependencies

Read the full contents of:

- kebos-backend/app/main.py
- kebos-backend/app/core/config.py
- qmind_enterprise/external_dataset_loader.py

Define and implement graceful degradation for every dependency.

Kebos must never have a total outage from a single dependency failure.

1. Create kebos-backend/app/core/degraded_mode.py:

```

from enum import Enum
from dataclasses import dataclass

```

```

class DependencyStatus(str, Enum):
    HEALTHY    = "healthy"
    DEGRADED   = "degraded"
    FAILED     = "failed"
    QUARANTINED = "quarantined"

@dataclass
class DegradedModePolicy:
    dependency: str
    status: DependencyStatus
    fallback_behaviour: str
    confidence_cap: float = 1.0    # cap QMind confidence in this degraded state

DEGRADED_POLICIES = {
    "abuseipdb":      DegradedModePolicy("abuseipdb",
DependencyStatus.FAILED,
                                "Use 7 remaining feeds, log reduced coverage",
0.85),
    "all_feeds_down": DegradedModePolicy("all_feeds", DependencyStatus.FAILED,
                                "CatBoost + UEBA only, cap confidence at 0.65",
0.65),
    "groq_api":      DegradedModePolicy("groq_api",
DependencyStatus.FAILED,
                                "Queue SOC reports, fall back to Jinja2 templates"),
    "vault":         DegradedModePolicy("vault",
DependencyStatus.FAILED,
                                "Read-only mode – no new sessions, existing valid
until expiry"),
    "kafka":         DegradedModePolicy("kafka",
DependencyStatus.FAILED,
                                "Direct HTTP to QMind fallback, no async pipeline"),
    "redis":         DegradedModePolicy("redis",
DependencyStatus.FAILED,
                                "DB-backed rate limit, JTI check via PostgreSQL
(slower)"),
    "qmind":         DegradedModePolicy("qmind",
DependencyStatus.FAILED,
                                "CatBoost only, cap confidence at 0.60"),
    "zeek":          DegradedModePolicy("zeek",
DependencyStatus.FAILED,
                                "Continue with agent-based telemetry only"),
    "ct_monitor":    DegradedModePolicy("ct_monitor",
DependencyStatus.FAILED,
                                "Reactive detection only, no proactive CT log

```

```
monitoring"),
    }
```

2. Create health check endpoint that surfaces degraded state:

GET /api/v1/health/detailed

Returns status of every dependency with current mode.

ADMIN-only in production.

3. In SOC report generator: Jinja2 template fallback when LLM is unavailable:

Create kebos-backend/app/reporting/soc_template.j2:

A structured template that produces a valid (if less narrative) SOC report using only structured data from QMind – no LLM required.

This ensures CERT-In 6-hour deadline can always be met even if Groq/Gemma is down.

4. Add dependency health monitoring background task:

```
async def monitor_dependencies():
    while True:
        await asyncio.sleep(60) # check every minute
        # Check each dependency
        # Update DEGRADED_POLICIES status
        # Log degraded state changes
        # Alert if critical dependency fails
    task = asyncio.create_task(monitor_dependencies(), name="dependency-health-
monitor")
    task.add_done_callback(handle_consumer_crash)
```

Show complete diff.

VALIDATION PROMPT (run after all phases — expanded from v2.0)

Read the full contents of ALL modified files across the entire project.

Perform a complete security and integration audit and verify every item below.

For each item, report: PASS / FAIL / NOT_FOUND with the file and line number.

If any CRITICAL item fails, describe exactly what code change is needed.

== AUTHENTICATION ==

[] JWT stored in HttpOnly SameSite=Strict cookie, NOT localStorage

[] JWT uses RS256 algorithm, NOT HS256

[] JWT expiry is exactly 15 minutes

- [] RSA private key loaded from Vault/environment, NOT hardcoded
- [] validate_environment() called in main.py before app startup
- [] Redis JTI blacklist checked on every authenticated request
- [] JTI keys namespaced by tenant_id: f"jti:{tenant_id}:{jti}"
- [] TOTP secrets encrypted via Vault transit, NOT plaintext in DB
- [] FIDO2 endpoints exist and are enforced for government tenants
- [] SessionRiskScorer runs on every authenticated request
- [] Impossible travel detection (>900 km/h auto-lock) implemented
- [] get_security_headers() registered as middleware

== PQC CRYPTOGRAPHY ==

- [] crypto_pq.py imports oqs and uses real liboqs calls
- [] Hybrid encryption: Kyber-768 + AES-256-GCM (not Kyber alone)
- [] USE_REAL_PQC=true in docker-compose for qmind service
- [] Dilithium-3 signing applied to audit log records
- [] PQC flag verified at startup before any customer-facing claim

== API SECURITY ==

- [] slowapi rate limiting uses Redis backend (NOT in-memory)
- [] Redis backend verified via isinstance check in startup
- [] CORS allow_headers does NOT contain "*"
- [] X-Request-ID middleware registered
- [] EgressControlledClient used for all outbound HTTP calls
- [] Egress allowlist enforced in STRICT_MODE

== QMIND INTEGRATION ==

- [] qmind service on port 8001 in docker-compose
- [] start.sh launches BOTH uvicorn AND kafka_consumer
- [] kafka_consumer.py uses AIOKafkaConsumer (not kafka-python)
- [] done_callback registered on every asyncio.create_task()
- [] kafka_consumer publishes to qmind.results after every /analyze
- [] Kebos backend consumes qmind.results topic
- [] update_threat_with_qmind_result() sets CONFIRMED_THREAT at >= 0.75
- [] /analyze always returns all 10 category_scores keys
- [] /signals/inject accepts all 8 source types including ct_log, paste_monitor
- [] SOCRreportGenerator uses JSON-mode prompting (not line-prefix parsing)
- [] LLM output validated against INJECTION_PATTERNS before use
- [] Both Groq and Gemma LLM clients wired at startup

== ZERO TRUST ARCHITECTURE ==

- [] 3 isolated Docker networks (frontend-net, app-net, data-net)
- [] No service has more networks than it needs
- [] Docker socket replaced by Tecnativa proxy in HoneyGrid
- [] All containers run as non-root user with no-new-privileges

- [] Syslog uses TCP+TLS port 6514 (TLSSyslogHandler), NOT UDP
- [] Vault PKI issues service certificates (or VAULT_PKI_ENABLED=False logged)
- [] Row-level security enabled on threat_events and audit_logs tables
- [] tenant_id present on all tables
- [] Redis keys namespaced by tenant_id throughout
- [] Kafka ACL script exists in docker/kafka-acls.sh

== DATA CLASSIFICATION ==

- [] LLMDDataSanitiser strips NEVER_EXTERNAL fields before external LLM calls
- [] CONFIDENTIAL data routed to local Gemma (never Groq/Gemini)
- [] Government tenant always uses local Gemma
- [] Data classification enforced before any /analyze call with customer data

== PROACTIVE INTELLIGENCE ==

- [] certstream CT log monitor starts at application startup
- [] CT monitor injects to QMind via /signals/inject with source="ct_log"
- [] Paste monitor runs every 30 minutes
- [] Domain monitor runs every 6 hours
- [] "Proactively detected" badge shown in UI for ct_log/crawler sources
- [] Supplier trust engine running against all 8 feeds
- [] Quarantined feeds fall back to signed cached data

== DECEPTION ==

- [] HoneyGridManager deployed on CONFIRMED_THREAT
- [] HoneyGrid connects via docker-proxy (NOT raw socket)
- [] Honeytoken endpoints exist (generate, list, trigger)
- [] Internal honeytokens checked on every request in main.py
- [] POST /api/deception/events calls /signals/inject with source="honeypot"
- [] kebos_deception_net defined with internal: true

== REPORTING ==

- [] CERT-In report signed with Dilithium-3
- [] Jinja2 template fallback exists for LLM-unavailable scenarios
- [] CERT-In SLA monitor running (alerts at 5-hour mark)
- [] Case auto-created on every CONFIRMED_THREAT with 6-hour deadline

== NTA + ENDPOINT ==

- [] Zeek log ingestor starts at startup (if ZEEK_ENABLED=True)
- [] Sysmon + auditd ingestion endpoints exist
- [] Vulnerability management import endpoint exists
- [] Vulnerability correlation with QMind CVE_Exploitation signals

== TIP + UEBA ==

- [] IOC records auto-created from QMind results


```
[ ] MITRE ATT&CK mapping endpoints exist
[ ] Threat actor profiles seeded (SideWinder, Lazarus, Bitter minimum)
[ ] UEBA baseline engine running and updating on each authenticated request
[ ] UEBA anomalies inject Insider_Threat signals to QMind
[ ] Analyst feedback endpoint publishes to analyst.feedback Kafka topic
[ ] Feedback consumer running as background task
```

== CASE MANAGEMENT + PLAYBOOKS ==

```
[ ] Cases auto-created on CONFIRMED_THREAT
[ ] cert_in_deadline set to NOW() + 6 hours on case creation
[ ] Playbook engine executes REVERSIBLE actions automatically
[ ] IRREVERSIBLE actions blocked pending Digital Twin + analyst approval
[ ] Approval workflow endpoints exist
[ ] Forensic timeline reconstruction endpoint exists
```

== MULTI-TENANCY ==

```
[ ] Tenant management endpoints exist (ADMIN only)
[ ] Tenant auth policy enforced per tenant_type
[ ] QMind thresholds configurable per tenant
[ ] All RLS policies active on all tenant-scoped tables
```

== RESILIENCE ==

```
[ ] Vault emergency rotation endpoint exists (ADMIN + FIDO2 required)
[ ] Emergency rotation flushes all JTI tokens
[ ] Graceful degradation policies defined for all 9 dependencies
[ ] Dependency health monitor running as background task
[ ] SOC report Jinja2 fallback tested independently of LLM availability
```

== SILENT FAILURE CHECKS (from v2.0 – must remain passing) ==

```
[ ] Skipping kafka consumer crash does NOT silently drop QMind results
[ ] slowapi is NOT using in-memory storage
[ ] _parse_report_sections does NOT use line-number prefix detection
[ ] DigitalTwinSimulator.simulate_action() is NOT a pass stub
[ ] update_threat_with_qmind_result() is NOT a pass stub
[ ] SOC LLM client is NOT None at runtime
[ ] Docker socket is NOT mounted directly (Tecnativa proxy only)
[ ] syslog is NOT using UDP
[ ] TOTP secrets are NOT stored plaintext
```

NOTES FOR WINDSURF CASCADE USAGE

1. **Always run the Context Injection block at the start of each new session.** Cascade has no

memory between sessions.

2. **Review every diff before approving.** For auth, crypto, PQC, ZTA, and LLM routing changes — review carefully. AI-generated security code is occasionally subtly wrong in ways that only appear under adversarial conditions.
3. **Prompt ordering is mandatory.** Every phase depends on the previous. 5.3 (RLS) depends on 5.1 (network split). 6.4 (supplier trust) depends on 2.1b (/signals/inject). Do not skip.
4. **After each prompt, run:** `pytest kebos-backend/tests/ -x docker-compose up --build` Verify health endpoint: `curl http://localhost:8000/health`
5. **Hardware-aware development modes:**
 - Auth/API work (Mode A): PostgreSQL + Redis + Kebos + Frontend only (~1.5GB)
 - QMind logic (Mode B): QMind + PostgreSQL only (~1GB)
 - Integration (Mode C): PostgreSQL + Redis + Kafka + Kebos + QMind (~3.5GB)
 - LLM (Mode D): Ollama + gemma:2b only — kill everything else first (~5-6GB)
 - Full stack (Mode E): AWS EC2 c6g.2xlarge or g4dn.xlarge only
6. **Three most dangerous silent failures (v2.0 — remain dangerous in v3.0):**
 - Skipping `done_callback` on `asyncio.create_task()`: consumer crashes silently
 - `slowapi` in-memory storage: rate limits bypass in multi-replica
 - Line-prefix LLM parsing: breaks on ~20% of real outputs
7. **Three new silent failures introduced in v3.0:**
 - CT monitor starts but `EgressControlledClient` blocks `certstream.calidog.io`: Add to `ALLOWED_EGRESS_DOMAINS` before enabling CT monitor
 - Supplier trust engine quarantines a feed but QMind still weights it at 1.0: Verify `get_qmind_weight()` is called in `external_dataset_loader.py`
 - UEBA baseline requires 7 days of data before scoring: In demos, seed baseline data in the test setup fixture
8. **Never commit until the Validation Prompt passes ALL CRITICAL checks.**
9. **Build sequence discipline:** Phase 1 complete → Phase 2 → ... → Phase 12 → Validation. The most common failure is building Phase 7 (frontend) before Phase 2 stubs are filled. Demo-readiness requires: Phase 1 + Phase 2 + Phase 3 + Phase 6.1 (CT logs) + basic Phase 7.

prompts), Multi-tenancy/Forensics (2 prompts), Supplier Trust/Resilience (2 prompts). Total: 17 prompts from v2.0 + 28 new prompts = 45 prompts in v3.0. All 60 product features accounted for. Build sequence validated.